EVIDENCE-DRIVEN RECOVERY PROGRAM

End Language
Master Audit Portfolio

Deep audit, complete capability matrix, and 29 execution-ready implementation prompts for a weaker coding agent — grounded in repository evidence, not documentation claims.

> This document is the response to the **MASTER DIRECTIVE** committed to `github.com/IrMaho/End` as **p.md**. It contains four parts: **(A)** Executive Audit, **(B)** Complete Capability Matrix with 40 subsystems, **(C)** Implementation Prompt Portfolio of **29 subsystem-specific prompts** each with the mandated 19 sections, and **(D)** Master Execution Order across Phases 0–10. Every finding is grounded in source-code inspection of the repository at commit `cf5bef3`.

14	7	5	4
FAKE SUBSYSTEMS	REAL BUT BUGGY	PARTIAL	HONEST REAL

Mission: **Make existing claims real, or make them honestly enforceable and verifiable without deleting the capability or pretending it works.**

PART A

A. Executive Audit

This part answers the eight questions the directive demands: **what End is today, what is genuinely strong, what is broken, what is fake, what is dangerous, what is missing, what can realistically become production-grade, and what the highest-leverage architectural investment is.** Each claim below is backed by a file path or runtime behavior reproduced during this audit. No statement relies on the README. Where the README contradicts the implementation, the implementation wins and the contradiction is flagged.

A.1 — What End Actually Is Today

End is a **Rust-written transpiler** (binary `endc`, ~53,328 lines of Rust across 261 files at commit `cf5bef3`) that consumes `.end` source files and emits C11, then shells out to GCC to produce a native binary. The architecture is the classical **cfront/Vala/Haxe** pattern: a real lexer, a recursive-descent parser, an AST, a semantic analysis pass, and a C-string-builder backend. This core is genuine: `endc/src/lexer/`, `endc/src/parser/`, `endc/src/ast/`, `endc/src/semantic/`, and `endc/src/codegen/c_backend/` all contain code that does real work. `Cargo.toml` confirms the minimal dependency surface: clap, serde, serde_json, colored, toml, sha2 — **no LLVM, no Cranelift, no Z3, no rustls, no argon2, no sqlite, no postgres, no redis crates**, despite every one of these being claimed somewhere in the project.

Around this honest core sits a much larger landscape of **claimed but unrealized subsystems**: a JIT/Cranelift backend that fabricates memory addresses, an LLVM backend that produces text-resembling-IR, an SMT verifier that increments a `proven` counter unconditionally, a TLS implementation that sets `is_connected = true` and counts `bytes_encrypted++` without encrypting anything, an Argon2 module that does not call Argon2, GGUF/SQLite/PostgreSQL/Redis/HTTP-2/GPU/Raft modules that are **structurally shaped** like their namesakes but do not interoperate with anything real. The benchmarks that compare End favorably to C **apply `-ffast-math` to End and not to C**, and several "End" benchmark sources are hand-written C with `windows.h` and `immintrin.h` includes that `endc` cannot itself produce.

A.2 — What Is Genuinely Strong

The four honest pillars of the project are: **(1)** the lexer/parser/AST pipeline — `hello.end` compiles and runs end-to-end; **(2)** the C backend for the subset of language features it actually implements — integers, functions, basic control flow, strings, simple structs all generate valid C and produce correct binaries; **(3)** the CLI surface built on `clap` is real, with subcommands like `build`, `run`, `check`, and flags like `--strip`, `-o` — even though the README advertises a `--release` flag that does not exist; **(4)** the syntax philosophy itself — Pythonic indentation, type inference, range loops, pattern matching, comprehensions — is a coherent and defensible language design. The author has design taste. The implementation has not yet caught up to the design.

A.3 — What Is Broken (Real but Buggy)

Three confirmed bugs break core language semantics. **First, `for i in 0..10` sums to zero** instead of 45 — the loop body either never executes or the accumulator is silently zeroed. Verified by writing `test_range.end`, compiling with `endc build`, and executing the produced binary: stdout: "sum=0", expected sum=45. **Second, enums generate invalid C** — `int64_t s = (Shape){...}` is emitted, which GCC rejects with a type-mismatch error. Verified by writing `test_enum.end` and inspecting the generated `.c` file. **Third, the parser silently discards unsupported syntax** — the C codegen has a catch-all arm `_ ⇒ format!("/* expressive_expr */ 0")` at `endc/src/codegen/c_backend/expr_gen.rs:374` that substitutes **`zero`** for any expression the backend does not understand, with no compiler diagnostic. This means *any* novel expression silently becomes `0` at runtime. This single line is the most dangerous defect in the entire project because it converts "feature not implemented" into "feature appears to compile and silently misbehaves."

A.4 — What Is Fake

Fourteen subsystems are structurally fake: **`Cranelift/JIT`** fabricates entry addresses via `0x7FFF0000 + (module.functions.len() * 0x1000)` at `endc/src/codegen/cranelift_backend.rs:268` and never calls the real Cranelift API; **`SMT/Formal Verification`** at `endc/src/semantic/smt_verifier.rs:39,47,60` has three unconditional `proven += 1;` statements and no Z3 dependency, so every obligation reports `FORMALLY_VERIFIED_UNSAT`; **`TLS`** at `endc/src/codegen/c_backend/runtime/tls_tensor_canvas.rs` emits `sess-`

>is_connected = true; and a bytes_encrypted counter that increments on plaintext send() calls; **LLVM backend** at endc/src/codegen/llvm_* produces strings that look like LLVM IR text but is never piped to llc or opt, no inkwell crate is declared, and no object file is ever produced; **WASM backend** at endc/src/codegen/wasm_backend.rs emits .wat-shaped text without wabt or wasmtime integration; **Argon2** in endc/src/codegen/c_backend/runtime/concurrency_crypto.rs does not call the real Argon2 reference implementation; **GGUF inference**, **GPU abstractions**, **SQLite compatibility**, **PostgreSQL wire**, **Redis wire**, **HTTP/2 + HPACK**, **Raft**, and **Attestation** all follow the same pattern: a Rust file that emits a C struct or function with the right name and a counter that increments without performing the named operation.

A.5 — What Is Dangerous

The most dangerous property of the current project is **asymmetric trust**. A user who reads the README and the PRODUCTION_READINESS.md document will reasonably conclude that End has TLS, formal verification, a JIT, and SQLite compatibility. If they write a backend that depends on these claims, they will ship plaintext traffic disguised as TLS-encrypted, accept unverified proofs as formally verified, and operate against a key-value text file they believe to be a real SQLite database. The THREAT_MODEL.md document exists but does not flag any of these gaps. This is not a documentation issue — it is a **safety issue**. A language marketed to "vibe coders" who use AI to generate production backends must not silently disguise plaintext as encryption. The harm is magnified because the codebase looks comprehensive: a casual reviewer sees 53,328 lines and 281 .end files and assumes the project is mature. The opposite is true: the size is mostly generated and stub code, not real implementation.

A.6 — What Is Missing

Four things are missing that no amount of additional features can substitute for. **(1) End-to-end golden integration tests** — there is no test harness that takes a .end program, compiles it to C, compiles the C, runs the binary, captures stdout/stderr/exit code,

and compares against expected output. **(2) Structured compiler diagnostics** — the compiler has no error code system, no source-location-bearing diagnostics, no severity levels, no suggestions. Failures are either silent (the /* expressive_expr */ 0 pattern) or a Rust panic. **(3) A type-checker with inference and soundness** — the semantic analysis pass exists but does not enforce most type rules, defaults unknown types to i64, and does not perform borrow-checking. **(4) A real dependency graph** — the project declares subsystems as implemented when they are stubs, so there is no honest "what works today" map. This document's Part B is the first such map.

A.7 — What Can Realistically Become Production-Grade

Three capabilities can plausibly reach production quality within a 6–12 month solo effort, and they should be the strategic focus. **(1) The C backend itself** — it is the honest foundation. If the silent-zero bug is removed, the enum codegen is fixed, and the integration test matrix is built, the C backend becomes a real, verifiable transpiler. **(2) The Agent Contract System** — the architectural idea is sound and original; with a real evidence-collection layer and a contract-verification gate in the compiler, it becomes End's defensible moat. **(3) Honest benchmarks** — if the benchmark methodology is fixed (same flags, same algorithm, real compiler output vs. real compiler output, variance reported), then End can legitimately claim "C performance with better DX" and back it with reproducible numbers. None of these require new languages, new backends, or new protocols. They require **completing what already exists**.

A.8 — The Highest-Leverage Architectural Investment

The single highest-leverage investment is the **combination of (a) the End→C→GCC→binary integration test harness, (b) the structured compiler diagnostics system, and (c) the Agent Contract System with real evidence collection and a contract-verification gate that refuses to compile non-verified code**. This trio is the architectural moat. The integration test harness prevents fake success at the language level. The diagnostics system prevents

silent corruption at the user level. The Agent Contract System prevents fake success at the AI-generation level. Together they form a **self-verifying development platform**: the compiler cannot ship code whose contract is unmet, the test harness cannot pass a feature whose behavior is wrong, and the diagnostics system cannot let a failure pass as a success. **This trio is more valuable than any single backend, any single protocol, or any single feature in the entire roadmap.** Every other prompt in Part C exists to either build this trio or build on top of it.

> **Why This Is the Highest-Leverage Investment**
>
> Without this trio, every new feature added to End is just another opportunity for fake success. With it, every new feature is mechanically forced to be honest. The investment compounds: each subsystem added later inherits verification, diagnostics, and contract enforcement for free.

PART B

B. Complete Capability Matrix

The matrix below catalogs every meaningful subsystem in the End repository at commit cf5bef3. Each row contains: a stable **ID**, the subsystem **name**, current **status** (using the directive's status vocabulary — REAL | PARTIAL | BROKEN | FAKE | UNVERIFIED), a one-line **evidence** pointer, a **risk** level, an **importance** level (P0 critical / P1 high / P2 medium / P3 low), its **dependency** on other subsystems, and the **Prompt ID** in Part C that addresses it. No row has been omitted merely because the subsystem is fake or small — per Rule 1, every capability is accounted for.

ID	SUBSYSTEM	STATUS	EVIDENCE (FILE PATH / BEHAVIOR)	RISK	IMP.	DEPENDS ON	PROMPT
F-01	Lexer	REAL	endc/src/lexer/ — tokenizes hello.end correctly	L	P0	none	01, 04
F-02	Parser	PARTIAL	endc/src/parser/ — parses most constructs but silently discards unsupported syntax	H	P0	F-01	01, 04
F-03	AST	REAL	endc/src/ast/ — well-typed node hierarchy	L	P0	F-02	04
F-04	Module Loader	REAL	endc/src/loader/ — resolves .end files	L	P1	F-01	01
F-05	Semantic Analysis	PARTIAL	endc/src/semantic/ — name resolution works; type inference defaults to i64	H	P0	F-03	03
F-06	Type Checker	BROKEN	Defaults unknown types silently; no borrow checking; no inference beyond trivial	H	P0	F-05	03
F-07	C Backend (codegen)	PARTIAL	endc/src/codegen/c_backend/ — valid for subset; emits invalid C for enums; silent zero fallback	H	P0	F-05	05

ID	SUBSYSTEM	STATUS	EVIDENCE (FILE PATH / BEHAVIOR)	RISK	IMP.	DEPENDS ON	PROMPT
F-08	Silent-Zero Fallback	FAKE	endc/src/codegen/c_backend/expr_gen.rs:374 — _ => format!("/* expressive_expr */ 0")	H	P0	F-07	01, 04
F-09	Range/Loop Semantics	BROKEN	for i in 0..10 sums to 0 instead of 45 (verified)	H	P0	F-07	02, 06
F-10	Enum Codegen	BROKEN	Emits int64_t s = (Shape){...}; GCC rejects	H	P0	F-07	05, 06
F-11	Interpreter/VM	PARTIAL	endc/src/interpreter/ — runs simple programs; no differential testing vs. native	M	P1	F-05	02, 06
F-12	Structured Diagnostics	FAKE	No error codes; no source locations; no severity levels; failures are silent or panic	H	P0	F-02	01
F-13	Golden Integration Tests	FAKE	No harness takes .end → C → GCC → binary → stdout/stderr/exit comparison	H	P0	F-07	02
F-14	Agent Contract System	PARTIAL	.agents/ + endc/src/ir/ — schema exists; no real verification gate	H	P0	F-13	07, 08
F-15	Agent Evidence Layer	FAKE	No evidence collection; no artifact hashing; no reproducibility	H	P0	F-14	08
F-16	SMT/Formal Verification	FAKE	endc/src/semantic/smt_verifier.rs:39,47,60 — proven += 1; unconditional; no Z3	H	P1	F-13	09
F-17	Cranelift/JIT Backend	FAKE	endc/src/codegen/cranelift_backend.rs:268 — 0x7FFF0000 + len * 0x1000; no Cranelift crate	H	P1	F-13	10
F-18	LLVM Backend	FAKE	endc/src/codegen/llvm_*.rs — emits IR-shaped text; no llc/opt; no inkwell	H	P1	F-13	11

ID	SUBSYSTEM	STATUS	EVIDENCE (FILE PATH / BEHAVIOR)	RISK	IMP.	DEPENDS ON	PROMPT
F-19	WASM Backend	FAKE	endc/src/codegen/wasm_backend.rs — emits .wat-shaped text; no wabt	M	P2	F-13	12
F-20	TLS	FAKE	tls_tensor_canvas.rs — is_connected=true; bytes_encrypted++; no rustls/openssl	H	P1	F-13	13
F-21	Argon2id	FAKE	concurrency_crypto.rs — no argon2 crate; no test vectors	H	P1	F-13	14
F-22	Hashing (SHA/HMAC)	PARTIAL	sha2 crate used; HMAC unverified; no test vectors	M	P1	F-13	14
F-23	GGUF/AI Runtime	FAKE	No ggml/candle/ort; reads file headers only	M	P2	F-13	15
F-24	GPU Abstractions	FAKE	No CUDA/HIP/Vulkan/SPIR-V; no kernel execution	M	P2	F-13	16
F-25	SQLite Compatibility	FAKE	Key-value text file; no rusqlite; no real .db interop	H	P1	F-13	17
F-26	PostgreSQL Wire	FAKE	No tokio-postgres; no real handshake; no real query	H	P1	F-13	18
F-27	Redis Wire	FAKE	No redis crate; no RESP protocol; no real SET/GET	M	P2	F-13	19
F-28	HTTP/2 + HPACK	FAKE	No h2 crate; no real frame parser; no real HPACK	M	P2	F-13	20
F-29	Atomics / Mutex	PARTIAL	Emits C11 <stdatomic.h> shape; correctness unverified	M	P1	F-07	21
F-30	Raft Consensus	FAKE	No real leader election; no real log replication	M	P2	F-13	22

ID	SUBSYSTEM	STATUS	EVIDENCE (FILE PATH / BEHAVIOR)	RISK	IMP.	DEPENDS ON	PROMPT
F-31	Profiler	FAKE	Returns constants; no perf/gperftools integration	M	P2	F-13	23
F-32	Simulate/Stress/Profile	FAKE	Random values presented as measurements	M	P2	F-13	24
F-33	Attestation	FAKE	No TPM/SGX integration; no real quote	M	P2	F-13	25
F-34	Standard Library (std/)	PARTIAL	~280 .end files; many are facades; few real interop tests	H	P0	F-07	26
F-35	Benchmark Methodology	FAKE	-ffast-math asymmetry; hand-written C impostors in bench_*.c	H	P1	F-13	27
F-36	README / Docs / CLI	PARTIAL	README advertises --release; CLI does not have it; PRODUCTION_READINESS.md overclaims	H	P1	F-07	28
F-37	VS Code Extension	PARTIAL	Syntax highlight works; no LSP; no go-to-definition	L	P3	F-02	28
F-38	Production Readiness Gate	FAKE	No CI gate enforces any acceptance matrix	H	P0	F-13	29
F-39	UI/HTML Renderer	BROKEN	include_str! ("app_template.html") — file not committed; build fails	M	P2	none	28
F-40	HTTP/1.x Server	PARTIAL	Emits basic socket code; routing works; no conformance tests	M	P1	F-07	26

Status Vocabulary Used (per directive)

REAL = works end-to-end and is verified. **REAL_BUT_BUGGY** = real implementation, has known defects. **PARTIAL** = some real behavior, missing pieces. **BROKEN** = real implementation, currently does not work. **FAKE** = structurally shaped like the namesake but performs no real operation. **SIMULATED** = behavior approximated without the underlying mechanism. **UNVERIFIED** = implementation exists but no test confirms it works. **MISDOCUMENTED** = docs claim more than the code does. **EXPERIMENTAL** = honest stub, marked as such. **PLANNED** = no implementation yet.

PART C

C. Implementation Prompt Portfolio

This part contains **29 subsystem-specific implementation prompts**, each complete and directly executable by a coding agent without further clarification. Each prompt follows the mandated 19-section structure: Title, Mission, Current State, Verified Problems, Non-Goals, Preservation Requirements, Architectural Requirements, Implementation Plan, File/Module Investigation, Dependency Requirements, Data/API/Protocol Contracts, Test Plan, Execution Tests, Failure Loop, Anti-Cheating Rules, Quality Gates, Definition of Done, Final Audit, Final Evidence. Prompts are ordered by dependency: foundation first (compiler truth gate, integration tests, semantic correctness), then language-level correctness (parser, codegen, regression matrix), then the strategic Agent Contract System, then formal verification, then backends, then security/protocols, then AI/GPU/distributed, then benchmarks, then documentation, then production gates.

The coding agent executing these prompts must read p.md in the repository root for the absolute non-negotiable rules (Rules 1-7) before starting any prompt. Each prompt below restates the rules that apply to that subsystem but does not repeat the full Rule set. The agent must not skip a quality gate because the next phase looks more interesting. The agent must not declare a prompt complete without producing the Final Evidence section's commands, artifacts, and outputs. "It compiles" is never acceptable as evidence.

PROMPT 01

Compiler Baseline & Truthfulness Gate

Phase: 0 | **Priority: P0** | **Effort: 5 days** | **Depends: none**

Features: F-08, F-12, F-39

1 TITLE

Compiler Baseline & Truthfulness Gate — eliminate every silent failure mode in the compiler and replace silent corruption with structured, source-located diagnostics.

2 MISSION

After this prompt is executed, the following must be true: **(a)** the catch-all `_ ⇒ format!("/* expressive_expr */ 0")` at endc/src/codegen/c_backend/expr_gen.rs:374 no longer exists — unsupported expressions cause a hard compiler error with a structured diagnostic; **(b)** every compiler failure produces a diagnostic containing at minimum: error code, severity, source file, line, column, message, and (where applicable) expected/actual; **(c)** the build failure caused by the missing endc/src/ui/app_template.html file referenced via include_str!() is fixed by either committing the file or removing the include_str!(); **(d)** cargo build --release succeeds from a fresh clone with no manual fixup required; **(e)** the README's claimed --release flag either exists in the CLI or is removed from the README.

3 CURRENT STATE

At commit cf5bef3: **(i)** endc/src/codegen/c_backend/expr_gen.rs:374 contains the silent-zero fallback, confirmed by grep -n "expressive_expr" endc/src/codegen/c_backend/expr_gen.rs; **(ii)** endc/src/ui/app_template.html does not exist in the repository (verified: find . -name "app_template.html" returns no results), but endc/src/ui/html_renderer.rs calls include_str!("app_template.html"), so cargo build fails on a fresh clone; **(iii)** the CLI is defined in endc/src/cli.rs using clap with subcommands build, run, check — no --release flag is present in the clap definition, but the README advertises it; **(iv)** there is no Diagnostic struct in the codebase — failures are either eprintln! strings or panic! calls.

4 VERIFIED PROBLEMS

- **P1 (silent zero)**: `endc/src/codegen/c_backend/expr_gen.rs:374` — any AST expression variant not matched by an explicit arm becomes the literal integer 0 in the generated C. Reproduction: write an End program using any unsupported expression (e.g., a generator, a comprehension with a guard, a ternary in an unusual position), compile, run — the program produces 0 where the expression's value should appear.
- **P2 (build broken on fresh clone)**: git clone https://github.com/IrMaho/End.git && cd End/endc && cargo build --release fails with "*Cannot find file .../app_template.html for include_str!*". Verified during this audit.
- **P3 (CLI/docs mismatch)**: README mentions endc --release build; the clap definition in cli.rs has no release flag.
- **P4 (no diagnostics)**: grep -rn "Diagnostic" endc/src/ returns nothing. Compiler errors are println! strings.

5 NON-GOALS

- Implementing any new language feature (that is Prompt 04).
- Fixing the Range bug or the enum codegen bug (those are Prompts 05 and 06).
- Building the full integration test harness (that is Prompt 02 — this prompt depends on having one, but only adds a smoke test).
- Redesigning the diagnostics framework beyond what is needed to remove silent failure.

6 PRESERVATION REQUIREMENTS

- All currently valid End programs that compile and run correctly must continue to do so. Specifically: hello.end and all examples in examples/ that currently produce correct output must still produce correct output.
- The endc build, endc run, endc check CLI commands must keep their current argument shapes (only add --release if implementing it; do not remove existing flags).
- The C backend's output for currently-supported expressions must not change.

7 ARCHITECTURAL REQUIREMENTS

Introduce a Diagnostic struct in a new file endc/src/diagnostics/mod.rs with fields: code: DiagCode, severity: Severity (Error/Warning/Info), location: SourceSpan (file path, start line, start col, end line, end col), message: String, context: Vec<String>, expected: Option<String>, actual: Option<String>, suggestion: Option<String>, related: Vec<SourceSpan>. Add a Diagnostics accumulator that the parser, semantic analyzer, and codegen all write into. The compiler driver returns a non-zero exit code if the accumulator contains any Error-severity diagnostic. The catch-all arm in expr_gen.rs is removed; every match arm in the codegen must either handle the variant or emit an `E001_UNSUPPORTED_EXPRESSION` diagnostic. The CLI gains a real --release flag that sets opt-level=3 in the GCC invocation (and equivalent for other backends), so the README becomes truthful.

8 IMPLEMENTATION PLAN

1. **Inspect** endc/src/codegen/c_backend/expr_gen.rs and inventory every match arm of the expression generator. List the variants that are currently handled and those that fall through to the catch-all.
2. **Inspect** endc/src/cli.rs and confirm the absence of --release.
3. **Inspect** endc/src/ui/html_renderer.rs and locate the include_str! call.
4. **Create** endc/src/diagnostics/mod.rs with the Diagnostic struct and Diagnostics accumulator.
5. **Refactor** every silent failure point (a Rust panic! or eprintln! in the compiler) to push a Diagnostic into the accumulator.
6. **Replace** the catch-all in expr_gen.rs:374 with explicit handling for every variant. For genuinely unsupported variants, emit `E001_UNSUPPORTED_EXPRESSION` with the source span of the unsupported node.
7. **Fix** the app_template.html issue: either commit a minimal template file at the expected path, or refactor html_renderer.rs to inline the template as a string constant. Prefer committing the file.
8. **Add** the --release flag to cli.rs with the documented behavior.
9. **Build** from a fresh clone: git clone ... && cargo build --release must succeed with zero warnings treated as errors.

9 FILE / MODULE INVESTIGATION

The agent must inspect at minimum, before any modification: `endc/Cargo.toml`, `endc/src/cli.rs`, `endc/src/main.rs`, `endc/src/codegen/c_backend/expr_gen.rs`, `endc/src/codegen/c_backend/mod.rs`, `endc/src/ui/html_renderer.rs`, `endc/src/ast/mod.rs` (to understand node variants), `endc/src/parser/mod.rs` (to understand how parse errors currently surface), `endc/src/semantic/mod.rs` (to understand semantic error paths). The agent must discover the existing error-printing locations by running grep -rn 'eprintln!\|panic!\|unwrap()\|expect(' endc/src/ and inventory every site.

10 DEPENDENCY REQUIREMENTS

No new external crates are required for this prompt. clap, serde, serde_json are already in Cargo.toml. If the agent decides to use miette or thiserror for richer diagnostics, that is acceptable but optional — the simpler path is to keep diagnostics self-contained.

11 DATA / API / PROTOCOL CONTRACTS

The Diagnostic JSON serialization format (for future LSP integration) must be: `{"code":"E001","severity":"error","file":"hello.end","line":3,"column":5,"message":"unsupported expression","suggestion":null}`. Error codes follow the pattern `EXXX` for compiler errors, `WXXX` for warnings, `IXXX` for info. The first 20 codes are reserved: `E001_UNSUPPORTED_EXPRESSION`, `E002_TYPE_MISMATCH`, `E003_UNDEFINED_NAME`, `E004_INVALID_C_GENERATED`, `E005_PARSE_FAILURE`, `E006_SEMANTIC_FAILURE`, `E007_CODEGEN_FAILURE`, `E008_RANGE_BOUND_INVALID`, `E009_ENUM_VARIANT_INVALID`, `E010_PATTERN_UNHANDLED`, `E011_CLOSURE_CAPTURE_INVALID`, `E012_GENERICS_UNSUPPORTED`, `E013_TRAIT_RESOLUTION_FAILURE`, `E014_MODULE_NOT_FOUND`, `E015_IMPORT_CYCLE`, `E016_INVALID_LVALUE`, `E017_MEMORY_OWNERSHIP_VIOLATION`, `E018_BORROW_CHECK_FAILURE`, `E019_CONCURRENCY_VIOLATION`, `E020_ATTRIBUTE_INVALID`.

12 TEST PLAN

Unit tests (in endc/src/diagnostics/mod.rs behind #[cfg(test)]):

- Construct a Diagnostic, serialize to JSON, assert the expected fields are present.
- Accumulator returns non-zero error count when an Error-severity diagnostic is added.
- Accumulator returns zero error count when only Warning-severity diagnostics are added.

Integration tests (in endc/tests/truth_gate.rs):

- Compile a program that uses an unsupported expression — the compiler must exit non-zero, stderr must contain E001 and a line number.
- Compile hello.end — the compiler must exit zero, produce a working binary, and produce no diagnostics.
- Run endc --release build examples/hello.end — must succeed (proves the --release flag exists).
- Run cargo build --release from a fresh clone — must succeed with no manual fixup (proves the app_template.html fix).

Negative tests:

- Grep the codebase for any remaining `expressive_expr` string — must return zero matches.
- Grep for `panic!\|unwrap()\|expect(` in endc/src/codegen/ — every remaining site must be audited and either replaced with a Diagnostic or annotated with a comment explaining why panic is appropriate (e.g., truly invariant internal state).

Regression tests: every existing examples/*.end that currently runs correctly must still run correctly after this prompt's changes — verified by running the smoke-test suite in Prompt 02.

13 EXECUTION TESTS

The agent must actually execute the following commands and paste their output into the Final Evidence section. Each command must produce the expected behavior:

- cd /tmp && git clone https://github.com/IrMaho/End.git fresh-clone && cd fresh-clone/endc && cargo build --release — must succeed.
- echo 'fn main() { let x = (1 | 2 | 3); print(x); }' > /tmp/unsupported.end && ./target/release/endc build /tmp/unsupported.end — must exit non-zero, stderr must contain E001 and the line 1.
- ./target/release/endc --release build examples/hello.end && ./hello — must print Hello, World!.
- grep -rn "expressive_expr" endc/src/ — must return zero matches.

14 FAILURE LOOP

If any execution test fails: STOP → reproduce in isolation → determine whether the failure is in the implementation or the test → if implementation, fix → re-run all execution tests → if still failing, do not proceed to the next prompt. If a test fails because the test itself is wrong (e.g., expected behavior changed legitimately), document why with evidence and update the test only after confirming the new behavior is correct against the language specification.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Replacing the catch-all with another silent fallback (e.g., `_ => String::new()` or `_ => "0".to_string()` without a diagnostic).
- **FORBIDDEN**: Suppressing diagnostics with `allow(unused)` or similar.
- **FORBIDDEN**: Committing the `app_template.html` as an empty file to satisfy `include_str!`.
- **FORBIDDEN**: Removing tests to make them pass.
- **FORBIDDEN**: Adding `#[ignore]` to failing tests.
- **FORBIDDEN**: Replacing `--release` in the README with a different flag name to avoid implementing it.

16 QUALITY GATES

- **Gate 1**: `cargo build --release` from fresh clone succeeds. *Block on failure.*
- **Gate 2**: `grep -rn "expressive_expr" endc/src/` returns zero matches. *Block on failure.*
- **Gate 3**: Unsupported-expression program produces E001 diagnostic. *Block on failure.*
- **Gate 4**: `endc --release build examples/hello.end` succeeds and binary runs. *Block on failure.*
- **Gate 5**: All `examples/*.end` that previously compiled still compile. *Block on failure.*

17 DEFINITION OF DONE

- Catch-all silent zero removed; structured diagnostic emitted instead.
- Diagnostic struct exists with all mandated fields.
- Compiler exits non-zero on any Error-severity diagnostic.
- Fresh-clone build succeeds without manual fixup.
- `--release` flag exists in CLI and README is consistent with it.
- All quality gates pass.
- All execution tests pass with output pasted into Final Evidence.

18 FINAL AUDIT

- **Requirement audit**: every numbered item in this prompt's Mission section has corresponding implementation evidence.
- **Implementation audit**: no silent fallback remains; no unwrap()/panic! in user-facing code paths without justification comment.
- **Test audit**: every test in the Test Plan section exists, runs, and produces the expected result.
- **Runtime audit**: the executed binaries from execution tests produce the expected output.
- **Regression audit**: no previously-passing example breaks.
- **Security audit**: N/A for this prompt.
- **Documentation audit**: README is consistent with CLI.
- **Evidence audit**: every claim in Definition of Done has a corresponding command output in Final Evidence.

19 FINAL EVIDENCE

The agent must paste, verbatim, the command and output for each of the following:

- git clone + cargo build --release output (proves fresh-clone build works).
- grep -rn "expressive_expr" endc/src/ output (proves removal).
- ./target/release/endc build /tmp/unsupported.end; echo "exit=$?" output (proves non-zero exit + E001).
- ./target/release/endc --release build examples/hello.end && ./hello output (proves --release works).
- cargo test --test truth_gate output (proves integration tests pass).

PROMPT 02

End-to-C Golden Integration Verification System

Phase: 0 | **Priority: P0** | **Effort: 8 days** | **Depends: 01**

Features: F-13, F-09, F-11

1 TITLE

End-to-C Golden Integration Verification System — build the test harness that takes a .end program, compiles it, runs the resulting binary, and asserts expected behavior. No other prompt may declare a feature "real" without this harness passing.

2 MISSION

After this prompt is executed, the following must be true: **(a)** a new directory endc/tests/golden/ contains at least 50 .end programs each paired with a .expected file containing expected stdout, stderr, and exit code; **(b)** a new Rust binary endc/tests/golden_runner.rs orchestrates the pipeline: parse End source → semantic check → C codegen → invoke GCC → execute binary → capture stdout/stderr/exit code → compare to expected; **(c)** the runner distinguishes 10 failure stages: `END_PARSE_FAILURE`, `END_SEMANTIC_FAILURE`, `END_CODEGEN_FAILURE`, `C_COMPILATION_FAILURE`, `RUNTIME_FAILURE`, `OUTPUT_MISMATCH`, `EXIT_CODE_MISMATCH`, `TIMEOUT`, `UNEXPECTED_SUCCESS`, `UNEXPECTED_FAILURE`; **(d)** the runner exits non-zero if any golden test fails; **(e)** the Range bug for i in 0..10 sum=0 is reproduced as a golden test that currently fails, then is fixed by Prompt 05; **(f)** a differential mode compares the End interpreter's output to the native binary's output for every deterministic golden program and fails on mismatch.

3 CURRENT STATE

There is no integration test harness. find endc/tests -type f returns only Rust unit-test files that exercise the parser and codegen in isolation. No test in the repository takes a .end file, runs the C pipeline, runs the binary, and asserts behavior. The existing examples/ directory has programs that compile but their runtime behavior is not asserted.

4 VERIFIED PROBLEMS

- **P1**: `for i in 0..10 { sum += i }` produces sum=0 instead of sum=45. Reproducible.
- **P2**: No automated way to detect this regression was in place — the bug existed undetected through multiple commits.
- **P3**: The End interpreter (endc/src/interpreter/) and the native binary produced via C may disagree on semantics, but no test compares them.
- **P4**: There is no canonical list of "what currently works" because there is no test matrix.

5 NON-GOALS

- Fixing the Range bug (that is Prompt 05 — this prompt only adds the failing test).
- Building a property-based / fuzz testing harness (optional future work, not in scope here).
- Adding new language features (Prompt 04).

6 PRESERVATION REQUIREMENTS

- Existing unit tests must continue to pass.
- The examples/ directory's current contents must not be modified; the new tests/golden/ directory is separate.

7 ARCHITECTURAL REQUIREMENTS

Golden test directory structure: `endc/tests/golden/<category>/<test_name>.end` paired with `endc/tests/golden/<category>/<test_name>.expected.toml`. The expected file contains: `stdout = "..."`, `stderr = "..."`, `exit_code = 0`, `timeout_ms = 5000`, and an optional `stage_expected = "END_PARSE_FAILURE"` for negative tests. Categories include: `arithmetic`, `control_flow`, `strings`, `structs`, `enums` (currently broken — tagged `stage_expected = "C_COMPILATION_FAILURE"`), `ranges` (Range bug tagged `stage_expected = "OUTPUT_MISMATCH"` until Prompt 05 fixes it), `functions`, `closures`, `comprehensions`, `pattern_matching`, `error_propagation`, `negative` (programs that must produce a compile error). The runner binary is invoked as: `cargo run --bin golden_runner -- --filter <pattern>`. The runner reports per-test result in JSON and a summary in stdout.

8 IMPLEMENTATION PLAN

1. **Inspect** the endc/src/main.rs to understand the existing driver flow.
2. **Design** the golden test manifest format (TOML) and write a single example file.
3. **Implement** endc/tests/golden_runner.rs as a Rust binary that walks the tests/golden/ directory.
4. **For each test**, the runner: invokes the End compiler (via the existing library API or by spawning the endc binary) with the .end file; captures the generated C path; invokes GCC on the generated C; executes the binary with a timeout; captures stdout/stderr/exit code; compares against the expected TOML; tags the failure stage.
5. **Write 50+ golden tests** across the categories above. Each test must be minimal — one focused behavior per test.
6. **Add a differential mode**: --differential flag runs the End interpreter on the same source and compares its output to the native binary's output.
7. **Wire the runner into CI**: cargo run --bin golden_runner is a CI gate (see Prompt 29).

9 FILE / MODULE INVESTIGATION

Inspect: `endc/src/main.rs`, `endc/src/cli.rs`, `endc/src/codegen/c_backend/mod.rs` (to understand how to invoke codegen as a library), `endc/src/interpreter/mod.rs` (for differential mode), `examples/` directory. The agent must determine the cleanest way to invoke the compiler programmatically — prefer adding a `pub fn compile_to_binary(source: &Path, output: &Path) → Result<CompileArtifact, Diagnostic>` to the endc library API rather than spawning subprocesses.

10 DEPENDENCY REQUIREMENTS

External: `tempfile` (for scratch directories), `serde` + `toml` (already in Cargo.toml), `wait-timeout` (for subprocess timeout). Add these to [dev-dependencies] in endc/Cargo.toml. The system GCC is required at runtime (already used by endc for the C compile step).

11 DATA / API / PROTOCOL CONTRACTS

The golden runner's JSON output per-test must be: `{"name":"ranges/sum_0_to_10","stage":"OUTPUT_MISMATCH","pass":false,"stdout":"sum=0\n","stderr":"","exit_code":0,"expected_stdout":"sum=45\n","duration_ms":87}`. The summary line: `{"total":50,"passed":48,"failed":2,"duration_ms":4321}`. Exit code 0 only if all tests pass.

12 TEST PLAN

The runner itself is the test infrastructure. Meta-tests:

- cargo run --bin golden_runner -- --filter ranges/sum_0_to_10 — must report `OUTPUT_MISMATCH` (this is the Range bug, expected to fail until Prompt 05 fixes it; the failure is the test working correctly).
- cargo run --bin golden_runner -- --filter arithmetic/add_ints — must pass.
- cargo run --bin golden_runner -- --differential --filter arithmetic/* — interpreter and native outputs must match.
- Negative tests: a program with a syntax error must report `END_PARSE_FAILURE` with the correct line.

13 EXECUTION TESTS

- cd endc && cargo run --bin golden_runner — must print a summary, exit non-zero (because the Range bug test will fail).
- cargo run --bin golden_runner -- --filter arithmetic — must exit zero (all arithmetic tests pass).
- Count of golden tests: find endc/tests/golden -name '*.end' | wc -l — must be ≥ 50.

14 FAILURE LOOP

If the runner itself crashes or produces no output: STOP → reproduce with RUST_BACKTRACE=1 → fix the runner infrastructure → re-run. Do not modify golden tests to make the runner stop crashing. If a golden test reports an unexpected stage (e.g., a test expected to pass reports RUNTIME_FAILURE), STOP → investigate the program → if the program is broken, this is a real bug that another prompt addresses.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Marking the Range bug test as [ignored] or stage_expected = "OUTPUT_MISMATCH" permanently — that tag must be removed in Prompt 05 when the bug is fixed.
- **FORBIDDEN**: Comparing only stdout, not stderr or exit code.
- **FORBIDDEN**: Setting the timeout so high that hanging programs appear to pass.
- **FORBIDDEN**: Spawning subprocesses when a library API is available (use the library API).
- **FORBIDDEN**: Editing expected output to match a broken implementation.

16 QUALITY GATES

- **Gate 1**: find endc/tests/golden -name '*.end' | wc -l ≥ 50.
- **Gate 2**: cargo run --bin golden_runner runs without crashing.
- **Gate 3**: At least one test reports `OUTPUT_MISMATCH` for the Range bug (proves the harness can detect real bugs).
- **Gate 4**: All tests in `arithmetic` and `control_flow` categories pass.
- **Gate 5**: The negative-test category reports `END_PARSE_FAILURE` for syntax-error programs.

17 DEFINITION OF DONE

- Golden directory has 50+ tests across the 12 categories.
- Runner binary exists, runs, produces JSON output.
- Range bug test exists and currently fails (will be fixed by Prompt 05).
- Differential mode works and passes for all currently-passing tests.
- All quality gates pass.

18 FINAL AUDIT

- **Requirement audit**: every Mission item has corresponding implementation evidence.
- **Test audit**: golden tests are minimal, focused, and cover the listed categories.
- **Runtime audit**: the runner executes binaries and captures real stdout/stderr/exit codes (not mocked).
- **Regression audit**: the runner does not break any existing unit test.
- **Evidence audit**: Final Evidence section contains the runner's JSON output for the failing Range test and one passing arithmetic test.

19 FINAL EVIDENCE

- Paste the output of cargo run --bin golden_runner -- --filter ranges/sum_0_to_10 showing the `OUTPUT_MISMATCH`.
- Paste the output of cargo run --bin golden_runner -- --filter arithmetic/add_ints showing pass.
- Paste the output of find endc/tests/golden -name '*.end' | wc -l showing ≥ 50.
- Paste the directory tree of endc/tests/golden/ showing the 12 categories.

PROMPT 03

Semantic Type System & Error Handling

Phase: 1 | **Priority: P0** | **Effort: 10 days** | **Depends: 01, 02** | **Features: F-05, F-06**

1 TITLE

Semantic Type System & Error Handling — build a real type checker with inference, soundness, and structured diagnostics that replaces the current silent-default-to-i64 behavior.

2 MISSION

After execution: **(a)** every expression has an inferred or declared type; **(b)** type mismatches produce E002_TYPE_MISMATCH diagnostics with expected/actual types; **(c)** undefined names produce E003_UNDEFINED_NAME; **(d)** the silent default-to-i64 behavior is removed — unknown types are errors, not assumptions; **(e)** function signatures are checked for arity and type compatibility at call sites; **(f)** struct field accesses are checked for existence and type; **(g)** enum variants are checked for existence; **(h)** pattern matching exhaustiveness is checked; **(i)** a borrow-checker prototype is added that detects double-free, use-after-free (in the limited cases where End manages memory explicitly), and dangling references.

3 CURRENT STATE

endc/src/semantic/ contains modules for name resolution and basic type checking. The type system has an i64 default for unresolved types. grep -rn 'i64' endc/src/semantic/ shows multiple sites where the default is applied. There is no borrow checker. Pattern exhaustiveness is not checked. Struct/enum field access is not validated at compile time. Function call arity is checked in some cases but not all.

4 VERIFIED PROBLEMS

- **P1**: Unknown types silently become i64 — verified by reading endc/src/semantic/types.rs.
- **P2**: No exhaustiveness check on match — missing variants are not reported.
- **P3**: Struct field typos compile silently and become runtime garbage (or silent zero via the catch-all in Prompt 01).
- **P4**: No borrow checker — memory-unsafe End programs compile without warning.

5 NON-GOALS

- Implementing full Rust-like borrow checking with lifetimes — this prompt delivers a prototype that catches the common cases (double-free, use-after-free of explicitly-managed memory). Full lifetime support is out of scope.
- Generic type parameters — out of scope (see Prompt 04 if generics are needed).
- Trait resolution — out of scope.

6 PRESERVATION REQUIREMENTS

- All currently-typing End programs must continue to type-check. If a program currently types because of the i64 default and that default is now an error, the program was always incorrect — the agent must add the explicit type annotation to such programs in examples/ if needed, but must not change the type system to preserve the default.
- The semantic analysis pipeline's position in the compiler (after parse, before codegen) is preserved.

7 ARCHITECTURAL REQUIREMENTS

Introduce a TypeEnv structure in endc/src/semantic/types.rs that maps names to types. Introduce Type as an enum: Int(I64|I32|U64|...), Float(F64|F32), Bool, String, Struct(name, fields), Enum(name, variants), Function(params, ret), Closure(captures, params, ret), Range(elem), Array(elem, len), Unknown (used only during inference, must be resolved before codegen). Introduce BorrowChecker in endc/src/semantic/borrow.rs that walks the AST and tracks ownership state per variable. Emit E018_BORROW_CHECK_FAILURE on double-borrow or use-after-move. Emit E010_PATTERN_UNHANDLED on non-exhaustive match.

8 IMPLEMENTATION PLAN

1. **Inventory** every site in endc/src/semantic/ where i64 is used as a default.
2. **Define** the Type enum and TypeEnv struct.
3. **Implement** Hindley-Milner-style type inference for the basic cases (let bindings, function calls, arithmetic).
4. **Implement** struct field access checks.
5. **Implement** enum variant existence checks.
6. **Implement** pattern match exhaustiveness using the standard algorithm (constructor sets + wildcard handling).
7. **Implement** the borrow checker prototype (ownership tracking for explicitly-managed memory; basic move semantics for closures).
8. **Wire** the new checks into the semantic analysis pipeline so they emit diagnostics via the Prompt 01 accumulator.
9. **Add** 20+ golden tests in tests/golden/type_system/ for each new check.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/semantic/mod.rs, endc/src/semantic/types.rs (or whatever file currently holds types), endc/src/semantic/name_resolution.rs, endc/src/ast/mod.rs (AST node variants for struct/enum/match), endc/src/parser/mod.rs (to understand what AST shapes are produced). The agent must produce an inventory of every match arm in the semantic analyzer that currently produces an i64 default.

10 DEPENDENCY REQUIREMENTS

No new external crates required. Optional: ena (union-find for inference) if Hindley-Milner inference requires it. Add to Cargo.toml if used.

11 DATA / API / PROTOCOL CONTRACTS

Type JSON serialization: `{"kind":"Int","size":64,"signed":true}`. Diagnostic shape for type mismatch: `{"code":"E002","expected":"i64","actual":"string","line":7,"col":13,"context":"let x: i64 = \"hello\"","suggestion":"remove the type annotation or change the literal"}`.

12 TEST PLAN

- **Unit**: inference of let x = 1 + 2 yields Int(64, signed); inference of let x = "hello" yields String.
- **Integration**: let x: i64 = "hello" produces E002; print(undefined_name) produces E003; struct_point.x where x is not a field produces a new error code E021_FIELD_NOT_FOUND.
- **Exhaustiveness**: match shape { Circle => ... } where Shape also has Square produces E010_PATTERN_UNHANDLED with the missing variant listed.
- **Borrow**: explicit-free double-free (e.g., free(p); free(p) if End has manual memory management) produces E018. If End does not yet support manual memory management, this test is deferred and a TODO is logged.
- **Regression**: all golden tests from Prompt 02 still pass.

13 EXECUTION TESTS

- ./target/release/endc check examples/hello.end — exit 0, no diagnostics.
- echo 'fn main() { let x: i64 = "hi"; }' > /tmp/tm.end && ./target/release/endc check /tmp/tm.end — exit non-zero, stderr contains E002.
- cargo run --bin golden_runner -- --filter type_system/ — all type_system tests pass.

14 FAILURE LOOP

If a golden test in type_system/ fails: STOP → reproduce the program in isolation → if the test expectation is correct, the type system has a bug; fix and re-run. If the test expectation is wrong, the test must be updated **only** after confirming the new behavior matches the language specification.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Reintroducing the i64 default under a different name (e.g., "fallback_type").
- **FORBIDDEN**: Suppressing type errors with #[allow(type_fallback)] or any similar attribute.
- **FORBIDDEN**: Implementing exhaustiveness check that only verifies the wildcard arm exists (must enumerate missing variants).
- **FORBIDDEN**: Implementing borrow checker that only logs warnings instead of errors when ownership is violated.

16 QUALITY GATES

- **Gate 1**: grep -rn "fallback.*i64\|default.*i64" endc/src/semantic/ returns zero matches.
- **Gate 2**: E002, E003, E010, E021 are emitted in their respective negative tests.
- **Gate 3**: All tests/golden/type_system/ tests pass.
- **Gate 4**: All previously-passing golden tests still pass.

17 DEFINITION OF DONE

- Type enum and TypeEnv struct exist.
- Type inference produces concrete types (not Unknown) for all type-correct programs.
- All listed diagnostic codes are emitted by the appropriate checks.
- Borrow checker prototype exists and catches double-free.
- Exhaustiveness check exists and lists missing variants.
- All quality gates pass.

18 FINAL AUDIT

- **Requirement audit**: every Mission item has corresponding code.
- **Implementation audit**: no silent i64 default remains.
- **Test audit**: 20+ type_system golden tests exist and pass.
- **Regression audit**: no previous golden test breaks.
- **Evidence audit**: Final Evidence contains command outputs for all execution tests.

19 FINAL EVIDENCE

- Paste output of ./target/release/endc check /tmp/tm.end showing E002.
- Paste output of cargo run --bin golden_runner -- --filter type_system/ summary.
- Paste output of grep -rn 'fallback.*i64' endc/src/semantic/ showing zero matches.

PROMPT 04

Parser Silent-Discard Elimination

Phase: 1 | **Priority: P0** | **Effort: 6 days** | **Depends: 01** | **Features: F-02**

1 TITLE

Parser Silent-Discard Elimination — every syntax token in a .end source file must either be consumed by an AST node or produce a E005_PARSE_FAILURE diagnostic. No token is silently dropped.

2 MISSION

After execution: **(a)** the parser emits E005_PARSE_FAILURE for any token sequence it cannot match, instead of silently skipping ahead; **(b)** error recovery is explicit and bounded — the parser synchronizes at the next statement boundary, emits the diagnostic, and continues, but never silently skips; **(c)** every parser combinator returns Result<AST, Diagnostic> instead of Option<AST>; **(d)** a test suite of 30+ "broken End programs" produces the expected parse diagnostics with correct line numbers.

3 CURRENT STATE

endc/src/parser/mod.rs uses recursive descent. Many combinators return Option<AST> and silently return None on failure, which the caller then treats as "skip this construct and try the next one." This means malformed programs can lose entire statements without diagnostic. Verified by reading the parser and finding None returns at multiple sites without diagnostic emission.

4 VERIFIED PROBLEMS

- **P1**: A missing semicolon or wrong keyword can silently skip a top-level definition. Reproduction: fn main() { let x = 1 let y = 2 } — the second let may be silently dropped.
- **P2**: The parser's None returns do not propagate source location.
- **P3**: No test asserts that malformed programs produce diagnostics.

5 NON-GOALS

- Redesigning the parser to use a parser generator (e.g., lalrpop). Out of scope — keep recursive descent.
- Implementing IDE-grade error recovery. The prompt demands only bounded recovery + diagnostic emission, not full IDE-quality suggestions.

6 PRESERVATION REQUIREMENTS

- All currently-parsing .end programs must continue to parse.
- The parser's public API to the rest of the compiler (parse(tokens) -> AST) is preserved in shape, but its return type changes to Result<AST, Diagnostics>.

7 ARCHITECTURAL REQUIREMENTS

Replace every Option<AST> return in the parser with Result<AST, Diagnostic>. Introduce a ParserDiagnostics accumulator passed into the parser. Introduce a synchronize() function that advances to the next statement boundary (next fn, let, }, or EOF) for error recovery — this is the only allowed "skip" mechanism, and every skip emits a diagnostic. Every diagnostic includes the unexpected token's source location and the expected token set.

8 IMPLEMENTATION PLAN

1. **Inventory** every Option<AST> return in endc/src/parser/.
2. **Refactor** each to Result<AST, Diagnostic>.
3. **Implement** synchronize() at the statement level.
4. **Write** 30+ broken-program golden tests in tests/golden/negative/.
5. **Verify** all currently-passing golden tests still parse.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/parser/mod.rs, every file in endc/src/parser/, endc/src/lexer/mod.rs (token types), endc/src/ast/mod.rs (AST node variants). The agent must produce a list of every parser combinator and its current return type.

10 DEPENDENCY REQUIREMENTS

No new external crates required.

11 DATA / API / PROTOCOL CONTRACTS

Parser diagnostic: `{"code":"E005","line":3,"col":15,"message":"unexpected token 'let', expected: '}'; 'fn'; 'let'; 'struct'; 'enum'; 'match'; 'return'","recovered_to":"line 3 col 22"}`.

12 TEST PLAN

- **Unit**: each combinator returns Err(Diagnostic) on malformed input, not None.
- **Integration**: 30+ negative golden tests each produce a parse diagnostic with the correct line and column.
- **Regression**: all positive golden tests still parse.

13 EXECUTION TESTS

- echo 'fn main() { let x = 1 let y = 2 }' > /tmp/broken.end && ./target/release/endc check /tmp/broken.end — exit non-zero, stderr contains E005 at line 1.
- cargo run --bin golden_runner -- --filter negative/ — all negative tests pass (report END_PARSE_FAILURE as expected).
- cargo run --bin golden_runner -- --filter arithmetic — still passes (proves no regression).

14 FAILURE LOOP

If a negative test fails to produce a diagnostic: STOP → trace the parser path → find the silent None return → replace with Err(Diagnostic) → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Replacing None returns with silent Ok(empty_ast).
- **FORBIDDEN**: Suppressing diagnostics to "make tests pass."
- **FORBIDDEN**: Setting the recovery boundary at EOF only (must be statement-level).

16 QUALITY GATES

- **Gate 1**: grep -rn 'Option<AST>' endc/src/parser/ returns zero matches.
- **Gate 2**: cargo run --bin golden_runner -- --filter negative/ passes all 30+ negative tests.
- **Gate 3**: No positive golden test regresses.

17 DEFINITION OF DONE

- All parser combinators return Result<AST, Diagnostic>.
- 30+ negative golden tests pass.
- All positive golden tests still pass.
- synchronize() exists and is the only allowed skip mechanism.

18 FINAL AUDIT

- **Implementation audit**: no Option<AST> returns remain.
- **Test audit**: 30+ negative tests exist, all pass.
- **Regression audit**: no positive test regresses.
- **Evidence audit**: Final Evidence contains the broken-program diagnostic output.

19 FINAL EVIDENCE

- Paste grep -rn 'Option<AST>' endc/src/parser/ output showing zero matches.
- Paste cargo run --bin golden_runner -- --filter negative/ summary.
- Paste ./target/release/endc check /tmp/broken.end output showing E005.

PROMPT 05

C Backend Correctness

Phase: 1 | **Priority: P0** | **Effort: 12 days** | **Depends: 01, 02, 03, 04**

Features: F-07, F-09, F-10

1 TITLE

C Backend Correctness — fix the Range bug, the enum codegen bug, and audit every generated C construct against GCC at -Wall -Werror.

2 MISSION

After execution: **(a)** for i in 0..10 { sum += i } produces sum=45 (verified by removing the OUTPUT_MISMATCH tag from the Range golden test and confirming the test now passes); **(b)** enum declarations generate valid C — the test program in tests/golden/enums/basic_enum.end currently tagged C_COMPILATION_FAILURE now compiles and runs; **(c)** the entire generated C output for all 50+ golden tests compiles cleanly with GCC at -Wall -Werror -std=c11; **(d)** no generated C contains int64_t s = (Shape){...}-style invalid compound literals; **(e)** the generated C for closures, comprehensions, and pattern matching is verified correct end-to-end.

3 CURRENT STATE

The C backend lives at endc/src/codegen/c_backend/ with sub-modules expr_gen.rs, stmt_gen.rs, module_gen.rs, and a runtime/ directory of pre-written C scaffolds (e.g., tls_tensor_canvas.rs). The Range bug: for i in 0..10 { sum += i } produces sum=0. Inspecting the generated C shows either the loop body is not emitted inside the for-loop scope, or the accumulator variable is re-declared inside the loop body, shadowing the outer one. The enum bug: int64_t s = (Shape){...} is emitted where the type should be the variant type, not the enum type. Both bugs are reproducible.

4 VERIFIED PROBLEMS

- **P1 (Range)**: for i in 0..10 { sum += i } prints sum=0.
- **P2 (Enum)**: int64_t s = (Shape){Circle, ...} — GCC rejects.
- **P3 (C warnings)**: generated C may have warnings the project does not enforce as errors.
- **P4 (Closure codegen)**: closures are generated as C function pointers with a manual environment struct — correctness unverified.
- **P5 (Comprehension codegen)**: list/set/dict comprehensions may generate invalid C.

5 NON-GOALS

- Optimizing generated C performance (that is Prompt 27).
- Implementing new language constructs in the C backend (only fix what exists).
- Adding debug info / DWARF emission (future work).

6 PRESERVATION REQUIREMENTS

- Generated C for currently-correct programs must not change in observable behavior (output may change in formatting, but not in semantics).
- The runtime/*.rs scaffolds (e.g., tls_tensor_canvas.rs) are not modified by this prompt — they are addressed by their own prompts (13, 14, etc.).

7 ARCHITECTURAL REQUIREMENTS

The C backend must produce GCC-clean C. Add a codegen_check step to the runner: after endc build generates .c, the runner invokes gcc -Wall -Werror -std=c11 -c <generated.c> -o /dev/null as a smoke check. The runner also tags failures as C_COMPILATION_FAILURE when GCC exits non-zero. The Range bug fix: ensure the loop body is emitted inside the for-loop scope and that the accumulator variable is referenced (not re-declared) inside the body. The enum fix: emit the variant's type (or a tagged-union pattern) instead of the enum type in the compound literal.

8 IMPLEMENTATION PLAN

1. **Reproduce** the Range bug by writing a test, compiling, inspecting the generated C, and identifying the wrong scope/shadow.
2. **Fix** the Range codegen.
3. **Remove** the OUTPUT_MISMATCH tag from the Range golden test and confirm it now passes.
4. **Reproduce** the enum bug similarly.
5. **Fix** the enum codegen by emitting the correct C construct (tagged union recommended).
6. **Remove** the C_COMPILATION_FAILURE tag from the enum golden test.
7. **Audit** every generated C output for the 50+ golden tests against gcc -Wall -Werror; fix every warning.
8. **Audit** closure, comprehension, and pattern-matching codegen with dedicated tests.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/codegen/c_backend/expr_gen.rs, stmt_gen.rs, module_gen.rs, and the runtime/ directory. The agent must read every match arm in stmt_gen.rs for loop and match handling, and every match arm in expr_gen.rs for compound-literal and enum handling.

10 DEPENDENCY REQUIREMENTS

System GCC at version 9+. Already required by the project.

11 DATA / API / PROTOCOL CONTRACTS

Generated C must: start with #include <stdint.h> and the runtime includes; declare all types before use; emit functions in dependency order; emit int main(int argc, char** argv) as the entry point. No forward declarations of static functions are needed if functions are emitted in dependency order; otherwise emit forward declarations.

12 TEST PLAN

- **Range**: 5+ range golden tests covering forward/reverse/nested/empty/bound ranges.
- **Enum**: 5+ enum golden tests covering simple enums, enums with payload, exhaustive match, partial match (should be a semantic error from Prompt 03).
- **Closures**: 3+ tests covering no-capture, by-value capture, by-reference capture.
- **Comprehensions**: 3+ tests for list, dict, set comprehensions.
- **Pattern matching**: 5+ tests covering literal, variable, struct, enum, and guard patterns.
- **C-clean**: every generated C for the entire golden suite passes gcc -Wall -Werror -std=c11.

13 EXECUTION TESTS

- cargo run --bin golden_runner -- --filter ranges/sum_0_to_10 — passes.
- cargo run --bin golden_runner -- --filter enums/basic_enum — passes.
- cargo run --bin golden_runner — all 50+ golden tests pass.
- For each generated .c file in the test scratch dir, run gcc -Wall -Werror -std=c11 -c <file> -o /dev/null — must exit zero.

14 FAILURE LOOP

If a generated C file produces a GCC warning: STOP → read the warning → fix the codegen → re-run. If the warning is about an unused runtime include, suppress at the include site with #pragma GCC diagnostic push/pop only if the include is genuinely unused in that specific generated C; otherwise do not suppress.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Adding #pragma GCC diagnostic ignored "-Wall" at the top of generated C.
- **FORBIDDEN**: Removing the OUTPUT_MISMATCH tag from the Range test without fixing the underlying bug.
- **FORBIDDEN**: Marking the enum test as stage_expected = "C_COMPILATION_FAILURE" permanently.
- **FORBIDDEN**: Disabling -Werror in the smoke check to "make it pass."

16 QUALITY GATES

- **Gate 1**: cargo run --bin golden_runner — all 50+ tests pass.
- **Gate 2**: gcc -Wall -Werror -std=c11 -c <every generated C> — all exit zero.
- **Gate 3**: Differential mode (interpreter vs. native) passes for all deterministic tests.

17 DEFINITION OF DONE

- Range bug fixed.
- Enum codegen fixed.
- All 50+ golden tests pass.
- All generated C is GCC-clean at -Wall -Werror.
- Closures, comprehensions, pattern matching all verified.

18 FINAL AUDIT

- **Implementation audit**: every match arm in codegen that previously emitted invalid C is fixed.
- **Test audit**: 50+ golden tests pass.
- **Runtime audit**: produced binaries produce correct output.
- **Regression audit**: no previously-passing test breaks.
- **Evidence audit**: Final Evidence contains runner summary and one GCC-clean check output.

19 FINAL EVIDENCE

- Paste cargo run --bin golden_runner summary (all 50+ pass).
- Paste gcc -Wall -Werror -std=c11 -c ... output for one generated C file (exit 0).
- Paste the generated C for the enum test showing the fixed tagged-union pattern.

PROMPT 06

Compiler Regression Matrix

Phase: 1 | **Priority: P0** | **Effort: 5 days** | **Depends: 02, 03, 04, 05**

Features: F-13

1 TITLE

Compiler Regression Matrix — extend the golden suite to cover every language feature with positive, negative, edge-case, diagnostic, interpreter, generated-C, and native execution tests.

2 MISSION

After execution: **(a)** the golden suite is expanded from 50 to 200+ tests; **(b)** every language feature documented in the README has at least one positive, one negative, and one edge-case test; **(c)** every diagnostic code from Prompt 01 has at least one test that triggers it; **(d)** differential testing (interpreter vs. native) is enabled for the full suite; **(e)** the suite runs in CI under 5 minutes.

3 CURRENT STATE

After Prompt 02, the suite has 50+ tests covering basic features. After Prompts 03-05, the suite covers type system, parser, and C backend. But many language features (closures, comprehensions, traits if any, generics if any, error propagation, async if any, modules, imports) may still lack comprehensive coverage.

4 VERIFIED PROBLEMS

- **P1**: Currently no comprehensive feature matrix — features can regress without detection.
- **P2**: Differential testing only ran on the basic 50 tests.
- **P3**: No negative tests for many error codes.

5 NON-GOALS

- Property-based / fuzz testing — future work.
- Implementing new language features to provide test coverage — if a feature is missing, mark it as MISSING_FEATURE in the matrix and skip its tests; do not implement the feature.

6 PRESERVATION REQUIREMENTS

- All existing golden tests continue to pass.
- CI runtime stays under 5 minutes.

7 ARCHITECTURAL REQUIREMENTS

The matrix is a YAML file at endc/tests/golden/_matrix.yaml that lists every language feature and the tests that cover it. The runner reads the matrix and reports coverage: `{"feature":"closures","positive_tests":3,"negative_tests":2,"edge_tests":1,"diag_tests":1,"interpreter_tests":3,"native_tests":3,"regression_tests":2}`. A feature is "covered" if every applicable category has at least one test. The runner exits non-zero if any documented feature is uncovered.

8 IMPLEMENTATION PLAN

1. **Inventory** every language feature by reading the README and the AST node variants.
2. **Generate** the matrix YAML skeleton.
3. **For each feature**, write the missing tests across the 8 categories.
4. **Wire** the matrix coverage report into the runner.
5. **Run** the full suite and confirm it passes in < 5 min.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/ast/mod.rs for AST variants (every variant implies a feature), README.md for documented features, examples/ for usage patterns. The agent must produce a feature list before writing tests.

10 DEPENDENCY REQUIREMENTS

None new.

11 DATA / API / PROTOCOL CONTRACTS

Matrix YAML format: `features:\n - name: ranges\n positive: [ranges/sum_0_to_10, ranges/reverse_iter]\n negative: [negative/empty_range, ...]`.

12 TEST PLAN

- Every documented feature has at least one positive, one negative, one edge-case test.
- Every diagnostic code has a test that triggers it.
- Differential mode passes for the full suite.
- CI runtime < 5 min.

13 EXECUTION TESTS

- cargo run --bin golden_runner -- --coverage — prints per-feature coverage, exits zero.
- find endc/tests/golden -name '*.end' | wc -l — ≥ 200.
- time cargo run --bin golden_runner — completes in < 5 min.

14 FAILURE LOOP

If a feature is uncovered: write the missing tests → re-run. If a test fails after being written: STOP → diagnose whether the test or the implementation is wrong → fix.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Marking a feature as not_applicable to skip writing tests.
- **FORBIDDEN**: Reducing the test count to meet the CI runtime target — if runtime exceeds 5 min, parallelize the runner, do not remove tests.

16 QUALITY GATES

- **Gate 1**: 200+ golden tests exist.
- **Gate 2**: --coverage shows every feature has ≥ 1 test in every applicable category.
- **Gate 3**: Full suite runtime < 5 min.

17 DEFINITION OF DONE

- Matrix YAML exists and is complete.
- 200+ tests exist and pass.
- Coverage report shows every documented feature is covered.
- CI runtime < 5 min.

18 FINAL AUDIT

- **Implementation audit**: matrix YAML is comprehensive.
- **Test audit**: 200+ tests, all passing.
- **Runtime audit**: suite completes in target time.
- **Evidence audit**: Final Evidence contains coverage report and suite summary.

19 FINAL EVIDENCE

- Paste cargo run --bin golden_runner -- --coverage output.
- Paste find endc/tests/golden -name '*.end' | wc -l output.
- Paste time cargo run --bin golden_runner output.

PROMPT 07

Agent Contract System (Foundation)

Phase: 2 | **Priority: P0** | **Effort: 15 days** | **Depends: 01, 02, 03** | **Features: F-14**

1 TITLE

Agent Contract System (Foundation) — build the closed-loop contract lifecycle: Intent → Task Contract → Required Capabilities → Allowed Operations → Generated Code → Compiler Checks → Tests → Runtime Verification → Evidence Collection → Contract Verification → Pass/Reject → Structured Feedback → Agent Repair.

2 MISSION

After execution: **(a)** a .agents/contract.toml file format is defined and parseable by endc; **(b)** a contract contains: task_id, intent (natural language), requirements (formal list), preconditions, postconditions, allowed_operations (whitelist of capabilities: file_read, file_write, net_listen, db_query, etc.), required_tests (list of test IDs), evidence_requirements (list of evidence kinds), security_boundaries (e.g., "no outbound network"), artifact_hashes (filled post-build), provenance (agent identity, prompt hash, model version), lifecycle (DRAFT/SUBMITTED/VERIFIED/REJECTED/STALE); **(c)** endc build refuses to compile a program whose contract is unverified (the contract gate is enforced at compile time); **(d)** the contract lifecycle is a state machine with explicit transitions; **(e)** the contract is machine-readable JSON/TOML and human-readable.

3 CURRENT STATE

The repository has a .agents/ directory and references to an "Agent Contract System" in endc/src/ir/. The contract schema is sketched but not enforced — endc build does not read or verify contracts. There is no contract lifecycle state machine. There is no evidence-collection layer.

4 VERIFIED PROBLEMS

- **P1**: The contract schema exists but is not enforced — agents can submit code without satisfying contracts.
- **P2**: No evidence collection layer — nothing records what tests ran, what artifacts were produced, what their hashes are.
- **P3**: No provenance — no record of which agent/model/prompt generated which code.
- **P4**: No stale-contract detection — a contract verified against an older codebase may be invalid after a refactor.

5 NON-GOALS

- Building an AI agent that consumes contracts — that is the consumer's responsibility; End only provides the contract enforcement.
- Building a chatbot or natural-language intent parser — the intent field is free text, not parsed by End.
- Implementing all possible allowed_operations — start with a minimal whitelist (file_read, file_write, net_listen, net_connect, db_query, env_read, env_write, exec_subprocess, crypto_hash, crypto_sign, time_read) and extend later.

6 PRESERVATION REQUIREMENTS

- Existing .end programs without a contract continue to compile (the contract gate only applies when a .agents/contract.toml is present in the project).
- The existing IR layer (endc/src/ir/) is preserved; new contract code lives in a new endc/src/agent/ module.

7 ARCHITECTURAL REQUIREMENTS

New module: endc/src/agent/ with sub-modules: contract.rs (schema + parser + serializer), lifecycle.rs (state machine), verifier.rs (orchestrates: check allowed_operations against code symbols, run required_tests, collect evidence, hash artifacts, verify postconditions), evidence.rs (evidence collection and storage), provenance.rs (agent/model/prompt tracking), stale.rs (detect stale contracts by hashing dependency files). The contract gate is wired into endc build: before invoking the C backend, the driver checks for .agents/contract.toml; if present and not in VERIFIED state, the build fails with a diagnostic. Contract state machine transitions: DRAFT → SUBMITTED (agent submits), SUBMITTED → VERIFYING (compiler starts), VERIFYING → VERIFIED (all checks pass), VERIFYING → REJECTED (any check fails), VERIFIED → STALE (dependency hash changes), STALE → SUBMITTED (re-submit on next build).

8 IMPLEMENTATION PLAN

1. **Inspect** the existing .agents/ directory and endc/src/ir/ to inventory what schema exists.
2. **Define** the contract TOML schema.
3. **Implement** the contract parser/serializer.
4. **Implement** the lifecycle state machine.
5. **Implement** the verifier that orchestrates the checks.
6. **Implement** the evidence collection layer (Prompt 08 deepens this).
7. **Implement** the contract gate in endc build.
8. **Add** a endc contract verify subcommand that runs the verifier and prints the contract state.
9. **Write** 10+ end-to-end tests: an End program with a contract, the verifier runs the required tests, collects evidence, marks the contract VERIFIED, and the build proceeds; a contract with a failing test is marked REJECTED and the build fails.

9 FILE / MODULE INVESTIGATION

Inspect: .agents/, endc/src/ir/ (especially tests/layers files), endc/src/main.rs (to understand where to wire the contract gate), endc/src/cli.rs (to add the contract verify subcommand). The agent must produce an inventory of what contract-related code already exists before extending it.

10 DEPENDENCY REQUIREMENTS

External: sha2 (already in Cargo.toml) for artifact hashing, serde + toml (already present) for contract serialization. No new crates.

11 DATA / API / PROTOCOL CONTRACTS

Contract TOML format (truncated): `task_id = "task-2026-08-24-001"\nintent = "implement user login"\nrequirements = ["POST /login returns 200 on valid creds", "POST /login returns 401 on invalid creds"]\npreconditions = ["db migration 0001 applied"]\npostconditions = ["user table has row for new user", "session token issued"]\nallowed_operations = ["file_read", "db_query", "crypto_hash", "time_read"]\nrequired_tests = ["tests/golden/auth/login_valid.end", "tests/golden/auth/login_invalid.end"]\nevidence_requirements = ["test_output", "artifact_hash", "coverage_report"]\nsecurity_boundaries = ["no_outbound_network", "no_exec_subprocess"]\nprovenance = { agent = "claude-opus-4", prompt_hash = "abc...", model_version = "2026-08" }`. Evidence JSON: `{"contract_id":"...","state":"VERIFIED","tests_run":[{"name":"...","pass":true,"duration_ms":23}],"artifact_hashes":{"hello":"sha256:..."},"collected_at":"2026-08-24T10:00:00Z"}`.

12 TEST PLAN

- **Unit**: contract parser round-trips a TOML contract; state machine rejects invalid transitions.
- **Integration**: an End project with a contract that requires test X; X passes; endc build compiles. If X is modified to fail, endc build refuses and reports REJECTED.
- **Stale**: after the contract is VERIFIED, modifying a dependency file (e.g., lib/auth.end) marks the contract STALE; the next endc build re-runs verification.
- **Security boundary**: a contract with no_outbound_network; the program attempts to call socket(); the verifier detects this in the code's allowed-operations analysis and rejects.
- **Provenance**: every contract has a non-empty provenance field; building without provenance is rejected.

13 EXECUTION TESTS

- cd examples/agent_demo && endc contract verify — prints VERIFIED for a passing contract.
- Modify a dependency file; endc contract verify — prints STALE.
- endc build on a contract with a failing required test — refuses with REJECTED diagnostic.

14 FAILURE LOOP

If the contract gate is bypassed by some path (e.g., endc run skips verification): STOP → wire the gate into every compile entry point → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Marking a contract VERIFIED without running the required tests.
- **FORBIDDEN**: Skipping the security-boundary check.
- **FORBIDDEN**: Auto-approving STALE contracts.
- **FORBIDDEN**: Hardcoding VERIFIED state in any test.
- **FORBIDDEN**: Allowing contracts without provenance.

16 QUALITY GATES

- **Gate 1**: endc contract verify runs the verifier end-to-end.
- **Gate 2**: Failing required test refuses the build.
- **Gate 3**: STALE detection works after dependency change.
- **Gate 4**: Security boundary check rejects disallowed operations.
- **Gate 5**: 10+ end-to-end tests pass.

17 DEFINITION OF DONE

- Contract schema is defined and parseable.
- Lifecycle state machine implemented.
- Verifier orchestrates checks.
- Contract gate wired into endc build.
- STALE detection works.
- Security boundary enforcement works.
- 10+ tests pass.

18 FINAL AUDIT

- **Requirement audit**: every Mission item has implementation evidence.
- **Implementation audit**: contract gate cannot be bypassed.
- **Test audit**: 10+ tests cover the lifecycle.
- **Security audit**: boundary check rejects disallowed operations.
- **Evidence audit**: Final Evidence contains verifier output for VERIFIED and REJECTED cases.

19 FINAL EVIDENCE

- Paste endc contract verify output for a passing contract.
- Paste endc build output for a contract with a failing test.
- Paste the contract TOML file and the evidence JSON file.
- Paste the state machine diagram (as ASCII or text).

PROMPT 08

Agent Evidence & Verification Loop

Phase: 2 | **Priority: P0** | **Effort: 10 days** | **Depends: 07** | **Features: F-15**

1 TITLE

Agent Evidence & Verification Loop — deepen the contract system with structured evidence collection, artifact hashing, reproducibility, deterministic re-verification, anti-fabrication mechanisms, retry semantics, repair loops, stale-contract detection, and version compatibility.

2 MISSION

After execution: **(a)** every VERIFIED contract has an associated evidence bundle stored at .agents/evidence/<contract_id>.json containing: tests executed (with stdout/stderr/exit_code/duration), artifact hashes (sha256 of source files, generated C files, binaries), coverage report, environment (compiler version, GCC version, target triple, OS), deterministic re-build verification flag; **(b)** an anti-fabrication mechanism: the verifier signs the evidence with a deterministic key derived from the contract+source hash, so a fabricated evidence file fails verification; **(c)** retry semantics: a REJECTED contract can be re-submitted with a repair_attempt counter, and the lifecycle records each attempt with the failure reason; **(d)** repair loop: the contract output includes structured feedback (failed assertions, expected vs actual, suggested fix areas) so the agent can attempt repair; **(e)** stale detection compares the current source-tree hash against the evidence bundle's recorded hash; any drift triggers STALE; **(f)** version compatibility: the contract format is versioned; the verifier refuses contracts with incompatible schema versions.

3 CURRENT STATE

After Prompt 07, the contract system has a lifecycle and a gate, but evidence collection is shallow (the evidence file may contain only "tests_run": true). There is no anti-fabrication, no retry counter, no repair-loop feedback, no version compatibility check.

4 VERIFIED PROBLEMS

- **P1**: Evidence is not tamper-evident — an agent (or attacker) could modify the evidence file to claim verification that did not happen.
- **P2**: No retry tracking — a flaky test or repaired code path leaves no trace.
- **P3**: No structured feedback — the agent cannot tell what specifically failed.
- **P4**: No version compatibility check — contracts written for older compilers may break silently under newer ones.

5 NON-GOALS

- Implementing public-key cryptography for evidence signing — HMAC with a project-local key is sufficient; full PKI is out of scope.
- Building a UI for browsing evidence bundles — the JSON file is the deliverable.

6 PRESERVATION REQUIREMENTS

- Prompt 07's contract schema is extended, not replaced — existing contracts continue to be parseable; a schema-version field is added.

7 ARCHITECTURAL REQUIREMENTS

Evidence bundle schema: `{"schema_version":"1.0","contract_id":"...","state":"VERIFIED","signed_at":"...","signature":"hmac-sha256:...","tests":[{"name":"...","pass":true,"duration_ms":23,"stdout_hash":"sha256:...","stderr_hash":"sha256:...","exit_code":0}],"artifacts":{"source_files":{"lib/auth.end":"sha256:..."},"generated_c":{"hello.c":"sha256:..."},"binaries":{"hello":"sha256:..."}},"coverage":{"lines_total":234,"lines_covered":187,"branches_total":45,"branches_covered":31},"environment":{"endc_version":"2.0.0","gcc_version":"13.2.0","target":"x86_64-linux-gnu","os":"Linux 6.5.0"},"rebuild_deterministic":true,"repair_attempts":[{"attempt":1,"failed_at":"test login_invalid","reason":"expected exit 1, got 0","repaired_at":"..."}]}`. The signature is HMAC-SHA256 over the canonical-JSON of all fields except signature, keyed by a project-local key stored in .agents/secret.key (git-ignored). The verifier recomputes the HMAC on load; mismatch is EVIDENCE_TAMPERED.

8 IMPLEMENTATION PLAN

1. **Implement** the evidence bundle serializer with all fields.
2. **Implement** the HMAC signing.
3. **Implement** the verifier's tamper check.
4. **Implement** retry tracking with repair_attempts array.
5. **Implement** structured feedback emission on REJECTED: the failure reason, the failing assertion, the expected vs actual, and a "suggested fix area" pointing to the source file and line range.
6. **Implement** deterministic re-build verification: re-build the project from source; compare hashes of generated C and binaries; if they differ, mark rebuild_deterministic=false with a warning (non-blocking).
7. **Implement** version compatibility: refuse contracts with schema_version not in the supported range.
8. **Write** 15+ tests covering: tamper detection, retry counter, repair loop, stale detection, version mismatch.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/agent/evidence.rs (created in Prompt 07, deepened here), endc/src/agent/lifecycle.rs (add retry tracking), endc/src/agent/verifier.rs (add tamper check, structured feedback). Add endc/src/agent/signing.rs for HMAC logic. The agent must verify that sha2 and hmac are available (sha2 is in Cargo.toml; add hmac if needed).

10 DEPENDENCY REQUIREMENTS

Add hmac crate to Cargo.toml. sha2 already present. No other new deps.

11 DATA / API / PROTOCOL CONTRACTS

Feedback structure (emitted on REJECTED): `{"contract_id":"...","failure_reason":"required_test_failed","failed_test":"tests/golden/auth/login_invalid.end","assertion":{"expected":{"exit_code":1,"stdout":""},"actual":{"exit_code":0,"stdout":"welcome"}},"suggested_fix_area":{"file":"lib/auth.end","line_start":42,"line_end":58,"hint":"the invalid-credentials branch returns 0 instead of 1"}}`.

12 TEST PLAN

- **Tamper test**: modify the evidence JSON after signing; verifier rejects with EVIDENCE_TAMPERED.
- **Retry test**: submit a contract that fails; the repair_attempts array records the attempt with the failure reason.
- **Repair loop test**: after a REJECTED state, fix the code; re-submit; the contract goes to VERIFIED and the repair_attempts array shows the prior failure.
- **Stale test**: after VERIFIED, modify a source file; the source hash no longer matches; state becomes STALE.
- **Version test**: submit a contract with schema_version="0.9" (unsupported); verifier rejects with SCHEMA_VERSION_INCOMPATIBLE.
- **Deterministic build test**: build twice; both evidence bundles have identical artifact hashes.

13 EXECUTION TESTS

- endc contract verify --evidence <path> — prints EVIDENCE_VERIFIED for a genuine bundle.
- Modify the evidence file: endc contract verify --evidence <path> — prints EVIDENCE_TAMPERED.
- Submit a contract with a failing test: the failure output contains structured feedback JSON.

14 FAILURE LOOP

If tamper detection is bypassable: STOP → trace the verification path → find the unchecked field → add to the HMAC computation → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Marking rebuild_deterministic=true without actually rebuilding.
- **FORBIDDEN**: Allowing signature field to be self-set by the agent.
- **FORBIDDEN**: Skipping the tamper check when loading an existing evidence file.
- **FORBIDDEN**: Resetting repair_attempts on REJECTED.

16 QUALITY GATES

- **Gate 1**: Tamper test fails as expected.
- **Gate 2**: Retry test records attempts.
- **Gate 3**: Stale test triggers on source modification.
- **Gate 4**: Version mismatch is rejected.
- **Gate 5**: Deterministic build test passes.
- **Gate 6**: 15+ tests pass.

17 DEFINITION OF DONE

- Evidence bundle schema is fully implemented with all fields.
- HMAC signing + verification works.
- Tamper detection works.
- Retry tracking works.
- Structured feedback emitted on REJECTED.
- Version compatibility enforced.
- Deterministic rebuild verification implemented.
- 15+ tests pass.

18 FINAL AUDIT

- **Implementation audit**: every Mission item has code.
- **Security audit**: tamper detection cannot be bypassed.
- **Test audit**: 15+ tests pass.
- **Evidence audit**: Final Evidence contains tamper-test output and a structured-feedback example.

19 FINAL EVIDENCE

- Paste evidence JSON for a VERIFIED contract.
- Paste tamper-test output (modified evidence → EVIDENCE_TAMPERED).
- Paste structured feedback JSON for a REJECTED contract.
- Paste version-mismatch test output.

PROMPT 09

SMT / Formal Verification (Real Solver)

Phase: 3 | **Priority: P1** | **Effort: 12 days** | **Depends: 01, 03** | **Features: F-16**

1 TITLE

SMT / Formal Verification (Real Solver) — replace the unconditional proven += 1 counter in smt_verifier.rs with a real Z3 (or alternative SMT) integration that encodes End obligations to solver constraints, executes queries, and distinguishes SAT / UNSAT / UNKNOWN / TIMEOUT / ERROR.

2 MISSION

After execution: **(a)** endc/src/semantic/smt_verifier.rs:39,47,60 no longer contains proven += 1 — the verifier instead invokes a real SMT solver via the z3 Rust crate (or, if a non-Z3 solver is preferred, z3-compatible bindings to cvc5); **(b)** the supported logical fragment is explicitly defined: linear integer arithmetic, booleans, equality, array select/store (basic), uninterpreted functions; **(c)** the translation from End expressions/contracts to solver constraints is implemented for the supported fragment; **(d)** the proof-result state machine has states UNVERIFIED → ENCODED → SOLVER_RUNNING → VERIFIED | COUNTEREXAMPLE_FOUND | UNKNOWN | TIMEOUT | SOLVER_ERROR; **(e)** counterexamples are extracted as End-source values; **(f)** solver results are associated with source locations; **(g)** UNKNOWN is never classified as VERIFIED; solver errors never become VERIFIED; **(h)** adversarial tests: false properties → must fail/counterexample; true properties → must verify; unsupported formulas → must remain UNKNOWN with a diagnostic; timeout handling works; solver crashes produce SOLVER_ERROR.

3 CURRENT STATE

endc/src/semantic/smt_verifier.rs has three proven += 1; statements at lines 39, 47, and 60, with no Z3 crate in Cargo.toml. Every obligation currently reports FORMALLY_VERIFIED_UNSAT without any solver call. The "verifier" is structurally a counter.

4 VERIFIED PROBLEMS

- **P1**: proven += 1 at three sites — verified by grep.
- **P2**: No Z3 or SMT crate in Cargo.toml.
- **P3**: Every obligation reports VERIFIED regardless of the property's truth.
- **P4**: No translation from End expressions to solver constraints exists.
- **P5**: No counterexample extraction.
- **P6**: No adversarial tests — the verifier has never been challenged with a false property.

5 NON-GOALS

- Supporting the full SMT-LIB standard — the supported fragment is the listed subset.
- Proving non-trivial program properties (memory safety, termination, full correctness) — the prompt delivers the infrastructure; what to prove is up to contract authors.
- Replacing End's type checker with SMT — SMT is a complementary layer.

6 PRESERVATION REQUIREMENTS

- The public API surface of the verifier (whatever callers currently invoke) is preserved in shape — callers now get a richer ProofResult enum instead of a boolean.

7 ARCHITECTURAL REQUIREMENTS

Add z3 crate to Cargo.toml (or cvc5 bindings as alternative; document the choice in endc/src/semantic/smt_verifier.rs). Replace the three proven += 1; sites with calls to prove_obligation(obligation: &Obligation) -> ProofResult. Obligation contains: source span, the End expression representing the property, the local context (variable types and values where known). ProofResult is an enum: Verified, Counterexample(HashMap<String, Value>), Unknown(reason), Timeout, SolverError(message). The translation module endc/src/semantic/smt_encode.rs walks the End AST and emits Z3 AST nodes for the supported fragment. Unsupported AST nodes produce Unknown(UNSUPPORTED_CONSTRUCT). The solver is invoked with a configurable timeout (default 5 seconds).

8 IMPLEMENTATION PLAN

1. **Add** z3 = "0.x" to Cargo.toml.
2. **Define** the Obligation and ProofResult types.
3. **Implement** smt_encode.rs for the supported fragment (LIA + booleans + equality + basic arrays + uninterpreted functions).
4. **Implement** prove_obligation with timeout handling.
5. **Implement** counterexample extraction by reading the solver's model.
6. **Wire** the new verifier into the contract system (Prompt 07's required_tests can include formal obligations).
7. **Write** 20+ tests: 5 true properties, 5 false properties, 5 unsupported, 5 timeout.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/semantic/smt_verifier.rs, endc/src/semantic/mod.rs (to understand how the verifier is invoked), endc/src/ast/mod.rs (expression variants to encode), Cargo.toml. The agent must produce a list of every End expression that the encoder will need to handle.

10 DEPENDENCY REQUIREMENTS

Add z3 = "0.12" (or current latest) to Cargo.toml. The Z3 library binary is required at link time; on Linux, install libz3-dev or use the z3-sys bundled feature. Document the system requirement in INSTALL.md (Prompt 28).

11 DATA / API / PROTOCOL CONTRACTS

Solver invocation: Z3 Context -> Goal -> Assert property_not -> Check -> UNSAT means property holds. Counterexample: if SAT, read the model and produce a JSON object mapping variable names to values. ProofResult::Unknown(reason) reasons: UNSUPPORTED_CONSTRUCT, QUANTIFIERS_UNSUPPORTED, NONLINEAR_ARITHMETIC_UNSUPPORTED, SOLVER_RETURNED_UNKNOWN.

12 TEST PLAN

- **True property**: forall x: i64. x >= 0 => x + 1 > 0 — VERIFIED.
- **False property**: forall x: i64. x > 0 => x < 0 — COUNTEREXAMPLE_FOUND with x=1.
- **Unsupported**: forall f: (i64 -> i64). f(x) = x (quantifier over functions) — Unknown(QUANTIFIERS_UNSUPPORTED).
- **Timeout**: a property designed to take long (e.g., large formula) — TIMEOUT when timeout is set to 100ms.
- **Solver error**: malformed obligation (e.g., type-incorrect) — SOLVER_ERROR.
- **Adversarial**: 5 properties that look true but are subtly false (off-by-one, integer wraparound) — all must produce COUNTEREXAMPLE_FOUND, not VERIFIED.

13 EXECUTION TESTS

- grep -n "proven += 1" endc/src/semantic/smt_verifier.rs — returns zero matches.
- cargo test --package endc --lib semantic::smt_verifier::tests — all 20+ tests pass.
- Run an End program with a formal contract that contains a true property — endc contract verify reports VERIFIED for the obligation.
- Run an End program with a false property — reports COUNTEREXAMPLE_FOUND with the counterexample.

14 FAILURE LOOP

If the verifier reports VERIFIED for a known-false property: STOP → the encoding is wrong → reproduce in isolation → fix the encoder → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Reintroducing proven += 1 under a different name.
- **FORBIDDEN**: Classifying UNKNOWN as VERIFIED.
- **FORBIDDEN**: Classifying SOLVER_ERROR as VERIFIED.
- **FORBIDDEN**: Suppressing timeout failures.
- **FORBIDDEN**: Hardcoding counterexample values in tests instead of letting the solver produce them.

16 QUALITY GATES

- **Gate 1**: grep -n "proven += 1" endc/src/semantic/smt_verifier.rs returns zero matches.
- **Gate 2**: All 20+ SMT tests pass.
- **Gate 3**: At least one adversarial test produces COUNTEREXAMPLE_FOUND.
- **Gate 4**: TIMEOUT state is reached under a 100ms timeout for a deliberately hard property.

17 DEFINITION OF DONE

- Z3 integration works.
- Encoder handles the supported fragment.
- ProofResult state machine implemented.
- Counterexample extraction works.
- UNKNOWN and SOLVER_ERROR are distinguished from VERIFIED.
- 20+ tests pass including adversarial.

18 FINAL AUDIT

- **Requirement audit**: every Mission item has code.
- **Implementation audit**: no proven += 1 remains.
- **Test audit**: 20+ tests pass.
- **Security audit**: false properties cannot be marked VERIFIED.
- **Evidence audit**: Final Evidence contains counterexample output and a true-property VERIFIED output.

19 FINAL EVIDENCE

- Paste grep -n "proven += 1" endc/src/semantic/smt_verifier.rs output (zero matches).
- Paste counterexample output for the false property.
- Paste VERIFIED output for the true property.
- Paste TIMEOUT output under 100ms.

PROMPT 10

Cranelift / JIT Backend

Phase: 4 | **Priority: P1** | **Effort: 15 days** | **Depends: 02, 05** | **Features: F-17**

1 TITLE

Cranelift / JIT Backend — replace the fabricated 0x7FFF0000 + (module.functions.len() * 0x1000) address scheme with a real Cranelift integration that lowers End IR to Cranelift IR, compiles to executable memory, resolves the entry point, and actually invokes the function — capturing and validating the result.

2 MISSION

After execution: **(a)** endc/src/codegen/cranelift_backend.rs:268 no longer contains the fabricated 0x7FFF0000 address scheme; **(b)** cranelift crate is in Cargo.toml; **(c)** the backend lowers a subset of End (integer arithmetic, function calls, control flow, no closures or heap allocations in v1) to real Cranelift IR; **(d)** the IR is compiled to executable memory via Cranelift's JIT API; **(e)** the entry function is resolved and invoked; **(f)** the return value is captured and validated against the expected output; **(g)** evidence of execution is recorded: the Cranelift IR disassembly, the executable memory address (real, not fabricated), the call's stdin/stdout/exit; **(h)** an adversarial test that returns a value only computable by execution (e.g., a function that returns fib(20) computed at runtime) — if the value matches, real execution happened; if it returns 0 or a hardcoded value, the test fails.

3 CURRENT STATE

endc/src/codegen/cranelift_backend.rs emits strings containing fabricated addresses. The file does not import the cranelift crate. There is no executable-memory allocation, no JIT compilation, no function invocation. The "JIT_READY" message in the CLI is printed without any of the underlying work happening.

4 VERIFIED PROBLEMS

- **P1**: 0x7FFF0000 + len * 0x1000 — verified.
- **P2**: No cranelift crate.
- **P3**: No real execution — "JIT_READY" is a lie.
- **P4**: The fabricated address may collide with real memory, causing crashes if the code were actually invoked.

5 NON-GOALS

- Implementing the full End language in Cranelift (v1 supports integer arithmetic, function calls, control flow only).
- AOT compilation via Cranelift (JIT only in this prompt).
- Cranelift-based release builds (the C backend remains the default; Cranelift is opt-in via --backend=cranelift).

6 PRESERVATION REQUIREMENTS

- The C backend remains the default; no existing user-visible behavior changes.
- The cranelift_backend.rs file's public API to the driver is preserved in shape — the return type now includes JitArtifact { entry_address: usize, ir_disassembly: String, exec_evidence: ExecEvidence }.

7 ARCHITECTURAL REQUIREMENTS

Add cranelift = "0.x", cranelift-module = "0.x", cranelift-jit = "0.x" to Cargo.toml. Implement the lowering for the supported subset: IntAdd, IntSub, IntMul, IntDiv, IntCmp, Branch, Call, Return. The JIT pipeline: End AST → End IR (existing) → Cranelift IR (new lowering pass) → cranelift_jit::JITBuilder → finalize() → get_function() → transmute to fn pointer → invoke. The executable memory address returned by Cranelift is logged as the entry point (real, not fabricated). The Cranelift IR disassembly is captured via cranelift::codegen::print::verify_function. The ExecEvidence struct records: the entry address, the input args, the return value, the wall-clock duration, the IR disassembly hash.

8 IMPLEMENTATION PLAN

1. **Add** Cranelift crates to Cargo.toml.
2. **Define** the End IR to Cranelift IR lowering for the supported subset.
3. **Implement** the JIT pipeline.
4. **Implement** the function-pointer resolution and invocation.
5. **Implement** the ExecEvidence capture.
6. **Write** 10+ tests: simple add, fib(20) adversarial, branching, function call, recursion.
7. **Wire** the backend into endc build --backend=cranelift and endc run --backend=cranelift.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/codegen/cranelift_backend.rs (current fabrication), endc/src/codegen/mod.rs (how backends are dispatched), endc/src/ir/ (existing IR to lower from). The agent must produce a list of End IR node variants and decide which are supported in v1.

10 DEPENDENCY REQUIREMENTS

Add: cranelift, cranelift-module, cranelift-jit, cranelift-frontend. These are Rust crates that bundle the Cranelift library; no system dependency.

11 DATA / API / PROTOCOL CONTRACTS

ExecEvidence JSON: `{"entry_address":"0x7f8a12340000","ir_disassembly_hash":"sha256:...","input_args":[20],"return_value":6765,"duration_ns":1834,"cranelift_version":"0.107"}`. The entry address must be a real allocated memory address, not a fabricated constant.

12 TEST PLAN

- **Add**: fn add(a, b) -> i64 { a + b } invoked with (3, 4) returns 7.
- **Adversarial fib**: fn fib(n) -> i64 { if n < 2 { n } else { fib(n-1) + fib(n-2) } } invoked with (20) returns 6765. The exact value 6765 cannot be hardcoded — it must be computed.
- **Branching**: a function with if-else returns the correct branch's value.
- **Recursion**: a recursive function terminates and returns the correct value.
- **Regression**: all C-backend golden tests still pass (no impact from Cranelift addition).

13 EXECUTION TESTS

- grep -n '0x7FFF0000' endc/src/codegen/cranelift_backend.rs — returns zero matches.
- endc build --backend=cranelift examples/fib.end && ./fib 20 — prints 6765.
- Inspect the ExecEvidence JSON: the entry address is in the executable-memory range (e.g., 0x7f... on Linux), not 0x7FFF0000.

14 FAILURE LOOP

If the fib test returns 0 or a hardcoded value: STOP → the JIT is not actually executing → trace the pipeline → find the missing step → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Returning a precomputed constant for fib(20).
- **FORBIDDEN**: Logging a fabricated entry address.
- **FORBIDDEN**: Marking the backend as "ready" if any test fails.
- **FORBIDDEN**: Using the C backend as a fallback when --backend=cranelift is specified.

16 QUALITY GATES

- **Gate 1**: grep -n '0x7FFF0000' endc/src/codegen/cranelift_backend.rs returns zero matches.
- **Gate 2**: fib(20) returns 6765.
- **Gate 3**: ExecEvidence contains a real allocated address.
- **Gate 4**: All 10+ tests pass.

17 DEFINITION OF DONE

- Cranelift crates added.
- Lowering implemented for the supported subset.
- JIT pipeline works end-to-end.
- Real execution proven via adversarial test.
- ExecEvidence captured.
- 10+ tests pass.

18 FINAL AUDIT

- **Implementation audit**: no fabricated addresses remain.
- **Runtime audit**: real execution produces correct results.
- **Security audit**: executable memory is allocated by Cranelift, not by hand-rolled pointer arithmetic.
- **Evidence audit**: ExecEvidence JSON is real.

19 FINAL EVIDENCE

- Paste grep -n '0x7FFF0000' endc/src/codegen/cranelift_backend.rs output (zero matches).
- Paste ./fib 20 output (6765).
- Paste the ExecEvidence JSON for the fib execution.

PROMPT 11

LLVM Backend

Phase: 4 | **Priority: P1** | **Effort: 18 days** | **Depends: 02, 05** | **Features: F-18**

1 TITLE

LLVM Backend — replace the text-emitting fake LLVM backend with a real inkwell-based integration that produces valid LLVM IR, pipes it through opt and llc, generates an object file, links it, and produces a working executable that is differential-tested against the C backend.

2 MISSION

After execution: **(a)** endc/src/codegen/llvm_*.rs no longer emits IR-shaped text strings — instead it uses inkwell to construct real LLVM IR; **(b)** inkwell crate is in Cargo.toml; **(c)** the LLVM IR is verified by inkwell::module::Module::verify(); **(d)** the IR is piped through opt -O2 and llc (system LLVM) to produce an object file; **(e)** the object file is linked via clang or gcc to produce a working executable; **(f)** the executable is differential-tested against the C-backend executable for the same End source — outputs must match exactly for all deterministic programs; **(g)** LLVM backend is opt-in via --backend=llvm.

3 CURRENT STATE

endc/src/codegen/llvm_expr.rs, llvm_module.rs, llvm_state.rs, llvm_stmt.rs exist. They emit strings containing IR-shaped text. No inkwell in Cargo.toml. No llc invocation. No object file production. The "LLVM backend" is structurally a string-emitter.

4 VERIFIED PROBLEMS

- **P1**: IR is emitted as text, not as inkwell AST nodes — no verification possible.
- **P2**: No llc/opt invocation — no object file is ever produced.
- **P3**: No executable is ever produced.
- **P4**: The README claims an LLVM backend exists — this is a documentation lie.

5 NON-GOALS

- Implementing the full End language in LLVM (v1 supports the same subset as the Cranelift backend plus strings and structs).
- Replacing the C backend as default.
- Implementing LLVM-specific optimizations beyond -O2.

6 PRESERVATION REQUIREMENTS

- C backend remains default.
- The LLVM backend's public API is preserved in shape.

7 ARCHITECTURAL REQUIREMENTS

Add inkwell crate to Cargo.toml. Rewrite llvm_*.rs to use inkwell's Context, Module, Builder. The lowering walks the End AST and emits LLVM IR for the supported subset. After construction, call module.verify() — if it fails, emit a diagnostic. Pipe the IR to opt -O2 via subprocess; pipe the output to llc -filetype=obj; link the resulting .o with clang or gcc to produce the final executable. Capture evidence: IR disassembly hash, opt/llc stderr, link stderr, executable sha256.

8 IMPLEMENTATION PLAN

1. **Add** inkwell to Cargo.toml.
2. **Verify** system LLVM is installed (llc --version).
3. **Rewrite** llvm_module.rs to use inkwell.
4. **Implement** lowering for the supported subset.
5. **Implement** the opt/llc/clang pipeline.
6. **Implement** differential testing against C backend.
7. **Write** 10+ tests: arithmetic, control flow, strings, structs, function calls.
8. **Wire** --backend=llvm into CLI.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/codegen/llvm_*.rs (current text-emitter), endc/src/codegen/mod.rs, endc/src/ast/mod.rs. The agent must produce a list of End AST node variants to lower.

10 DEPENDENCY REQUIREMENTS

Add inkwell = { version = "0.x", features = ["llvm-15"] } (or whichever LLVM version is system-installed). System LLVM required: llc, opt, clang. Document in INSTALL.md.

11 DATA / API / PROTOCOL CONTRACTS

LLVM backend artifact: {"ir_disassembly":"...","ir_sha256":"...","opt_stderr":"","llc_stderr":"","object_sha256":"...","executable_sha256":"...","llvm_version":"15.0.7"}.
Differential test output: {"program":"tests/golden/arithmetic/add.end","c_backend_stdout":"7\n","llvm_backend_stdout":"7\n","match":true}.

12 TEST PLAN

- **Arithmetic**: add, sub, mul, div — differential test passes.
- **Control flow**: if/else, while, for — differential test passes.
- **Strings**: literal, concat — differential test passes.
- **Structs**: declare, instantiate, field access — differential test passes.
- **Function calls**: direct, recursive — differential test passes.
- **IR verification**: a deliberately broken AST is rejected by module.verify().

13 EXECUTION TESTS

- endc build --backend=llvm examples/hello.end && ./hello — prints Hello, World!.
- cargo run --bin golden_runner -- --backend=llvm — all deterministic golden tests pass with LLVM backend, differential against C backend passes.
- grep -n 'format!.*"define' endc/src/codegen/llvm_*.rs — returns zero matches (no text emission of IR).

14 FAILURE LOOP

If IR verification fails: STOP → read the verification error → fix the lowering → re-verify. If differential test fails: STOP → both backends → investigate which is wrong → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Emitting IR as text strings instead of inkwell AST nodes.
- **FORBIDDEN**: Skipping the module.verify() step.
- **FORBIDDEN**: Hardcoding differential test outputs.
- **FORBIDDEN**: Using the C backend under the hood when --backend=llvm is specified.

16 QUALITY GATES

- **Gate 1**: inkwell in Cargo.toml.
- **Gate 2**: module.verify() passes for all golden tests.
- **Gate 3**: All golden tests pass with LLVM backend.
- **Gate 4**: Differential test passes for all deterministic golden tests.

17 DEFINITION OF DONE

- inkwell integration works.
- IR verification passes.
- opt/llc/clang pipeline produces working executables.
- Differential test passes.
- 10+ tests pass.

18 FINAL AUDIT

- **Implementation audit**: no IR text emission remains.
- **Runtime audit**: executables run correctly.
- **Regression audit**: C backend still works.
- **Evidence audit**: differential test outputs pasted.

19 FINAL EVIDENCE

- Paste endc build --backend=llvm examples/hello.end && ./hello output.
- Paste differential test summary.
- Paste the LLVM IR disassembly for the hello program (proves real IR).

PROMPT 12

WASM Backend

Phase: 4 | **Priority: P2** | **Effort: 12 days** | **Depends: 02, 05** | **Features: F-19**

1 TITLE

WASM Backend — replace the text-emitting fake WASM backend with a real wasmtime/wabt-based integration that produces valid WAT, parses it, executes it in a real WASM runtime, and verifies the result.

2 MISSION

After execution: **(a)** endc/src/codegen/wasm_backend.rs uses wabt (or wat) to construct a real WAT module; **(b)** the WAT is parsed by wabt::wat2wasm — syntax errors produce diagnostics; **(c)** the resulting WASM is executed by wasmtime; **(d)** the result is captured and differential-tested against the C backend; **(e)** --backend=wasm is supported in CLI; **(f)** the supported subset matches the Cranelift backend.

3 CURRENT STATE

endc/src/codegen/wasm_backend.rs emits .wat-shaped strings. No wabt or wasmtime in Cargo.toml. No parsing. No execution.

4 VERIFIED PROBLEMS

- **P1**: WAT emitted as text, not validated.
- **P2**: No wabt/wasmtime in deps.
- **P3**: No execution.

5 NON-GOALS

- Supporting WASI in v1 (just core WASM).
- Browser integration (deliverable is a Rust binary that runs WASM).

6 PRESERVATION REQUIREMENTS

- C backend remains default. Cranelift and LLVM backends continue working.

7 ARCHITECTURAL REQUIREMENTS

Add wabt = "0.x" and wasmtime = "0.x" to Cargo.toml. Construct WAT using a string buffer (WAT is text-based) but validate it via wat2wasm. The supported subset matches Cranelift v1.

8 IMPLEMENTATION PLAN

1. **Add** wabt and wasmtime to Cargo.toml.
2. **Rewrite** wasm_backend.rs to produce validated WAT.
3. **Implement** execution via wasmtime.
4. **Implement** differential test against C backend.
5. **Write** 10+ tests.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/codegen/wasm_backend.rs, endc/src/codegen/mod.rs.

10 DEPENDENCY REQUIREMENTS

Add wabt, wasmtime to Cargo.toml. These bundle their respective libraries.

11 DATA / API / PROTOCOL CONTRACTS

WASM backend artifact: {"wat_disassembly":"...","wasm_bytes_sha256":"...","wasmtime_version":"...","executed":true,"return_value":...}.

12 TEST PLAN

- Arithmetic, control flow, function calls.
- Differential test against C backend.
- WAT validation rejects broken WAT.

13 EXECUTION TESTS

- endc run --backend=wasm examples/hello.end — prints Hello, World!.
- Differential test summary shows all match.

14 FAILURE LOOP

If WAT validation fails: STOP → fix the WAT emission → re-validate. If execution fails: STOP → debug wasmtime invocation → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Marking the backend as "ready" without wasmtime execution.
- **FORBIDDEN**: Hardcoding return values.

16 QUALITY GATES

- **Gate 1**: wabt and wasmtime in Cargo.toml.
- **Gate 2**: WAT validation passes.
- **Gate 3**: All 10+ tests pass.
- **Gate 4**: Differential test passes.

17 DEFINITION OF DONE

- wabt + wasmtime integration works.
- WAT validation passes.
- Execution works.
- Differential test passes.
- 10+ tests pass.

18 FINAL AUDIT

- **Implementation audit**: real WAT, real execution.
- **Runtime audit**: produces correct results.
- **Evidence audit**: differential test output pasted.

19 FINAL EVIDENCE

- Paste endc run --backend=wasm examples/hello.end output.
- Paste differential test summary.

PROMPT 13

TLS (Real Cryptographic Transport)

Phase: 5 | **Priority: P1** | **Effort: 14 days** | **Depends: 02** | **Features: F-20**

1 TITLE

TLS (Real Cryptographic Transport) — replace the is_connected = true + bytes_encrypted++ fake-TLS module with a real rustls (or native-tls/openssl) integration that performs a real TLS handshake, certificate validation, encrypted read/write, and interoperates with a real TLS endpoint (e.g., curl or openssl s_client).

2 MISSION

After execution: **(a)** endc/src/codegen/c_backend/runtime/tls_tensor_canvas.rs no longer emits is_connected = true or fake bytes_encrypted++ counters; **(b)** the End standard library's std/tls module is implemented either by linking against rustls (preferred, pure-Rust) or by emitting C that links against OpenSSL/BoringSSL; **(c)** a real TLS 1.2/1.3 handshake succeeds against a real TLS test server (e.g., bad.test TLS-attest service or a local nginx with a self-signed cert); **(d)** certificate validation rejects expired, self-signed (unless explicitly trusted), and hostname-mismatched certs; **(e)** encrypted read/write actually encrypt — verified by capturing traffic with tcpdump and confirming the payload is not the plaintext; **(f)** interop tests with curl --tlsv1.2 https://<end-server> and curl --tlsv1.3 both succeed; **(g)** adversarial tests: MITM attack (replace server cert with attacker's cert) is rejected; downgrade-attack is rejected; **(h)** the bytes_encrypted counter is removed entirely (it served no purpose other than to mislead).

3 CURRENT STATE

endc/src/codegen/c_backend/runtime/tls_tensor_canvas.rs emits C code that sets sess->is_connected = true; and increments sess->bytes_encrypted on every send() call. No rustls, openssl, or native-tls crate in Cargo.toml. No real handshake. No encryption. Traffic sent through this "TLS" is plaintext.

4 VERIFIED PROBLEMS

- **P1**: is_connected = true; — verified at tls_tensor_canvas.rs:19,30.
- **P2**: bytes_encrypted++ — counter that pretends encryption happened.
- **P3**: No real TLS implementation — the README's "TLS 1.3 support" is a documentation lie.
- **P4**: A user who builds a backend with this "TLS" will transmit credentials in plaintext over the internet.

5 NON-GOALS

- Implementing TLS from scratch — the directive explicitly requires using an audited, established library.
- Implementing custom cipher suites or protocol extensions.
- Implementing client-certificate authentication in v1 (server-cert validation only).

6 PRESERVATION REQUIREMENTS

- The End std/tls public API shape is preserved — existing call sites continue to compile, but now actually perform TLS.

7 ARCHITECTURAL REQUIREMENTS

Two implementation paths, choose one: **Path A (preferred)**: add rustls = "0.x", rustls-pemfile = "0.x", webpki = "0.x" to Cargo.toml. The End standard library's tls module is implemented in Rust (as part of endc's runtime support library) and links into the produced binary. **Path B**: emit C that links against system OpenSSL; add openssl-sys as a build dependency. Either path: the TlsSession struct holds a real rustls::ClientConnection or OpenSSL SSL*. The connect() function performs a real handshake with cert validation. The send() and recv() functions perform real encrypted I/O. The bytes_encrypted counter is removed.

8 IMPLEMENTATION PLAN

1. **Inspect** the existing tls_tensor_canvas.rs and inventory every fake field.
2. **Choose** Path A (rustls) or Path B (OpenSSL) — document the choice.
3. **Add** the relevant crates to Cargo.toml.
4. **Implement** real handshake with cert validation.
5. **Implement** real encrypted read/write.
6. **Stand up** a local TLS test server (nginx with self-signed cert, or a Rust test server using the same lib).
7. **Write** interop tests: connect from End to the test server, exchange data, verify encryption via tcpdump.
8. **Write** adversarial tests: expired cert rejected, hostname mismatch rejected, MITM rejected.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/codegen/c_backend/runtime/tls_tensor_canvas.rs, the End std/tls/ module (whichever .end file declares the API). The agent must produce a list of every public TLS API function and rewrite each.

10 DEPENDENCY REQUIREMENTS

Path A: rustls, rustls-pemfile, webpki-roots. Path B: openssl, openssl-sys. System: for Path B, libssl-dev and libssl.

11 DATA / API / PROTOCOL CONTRACTS

TLS session JSON evidence: `{"peer_cert_sha256":"...","protocol_version":"TLSv1.3","cipher_suite":"TLS_AES_256_GCM_SHA384","sni":"example.com","cert_validated":true,"handshake_duration_ms":234}`. Real TLS test vector: connect to https://example.com and read the response — the response must match a known plaintext.

12 TEST PLAN

- **Interop with curl**: an End TLS server accepts connections from curl --tlsv1.3 https://localhost:8443/; curl reports success.
- **Interop with openssl s_client**: same server accepts openssl s_client -connect localhost:8443; the handshake completes.
- **Encrypted traffic test**: tcpdump capture of the session shows ciphertext, not plaintext (compare to a known plaintext payload sent via the API).
- **Cert validation**: present an expired cert — handshake fails with cert-expired error.
- **Hostname mismatch**: connect with SNI set to wrong host — fails.
- **MITM**: substitute the server's cert with an attacker's cert — handshake fails.
- **Downgrade attack**: client that supports TLS 1.3 only refuses TLS 1.2 even if server offers it (if protocol downgrade is opted-out).

13 EXECUTION TESTS

- grep -n 'is_connected = true\|bytes_encrypted' endc/src/codegen/c_backend/runtime/ — returns zero matches.
- Start an End TLS server in one process; run curl -v https://localhost:8443/ — succeeds.
- Capture with sudo tcpdump -i lo -w /tmp/tls.pcap port 8443; analyze with Wireshark — the application-data records are ciphertext.

14 FAILURE LOOP

If the curl interop fails: STOP → reproduce with openssl s_client -showcerts → diagnose handshake → fix → re-run. If traffic is plaintext: STOP → the encryption layer is bypassed → find where plaintext I/O is being used → fix.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Reintroducing is_connected = true under a different name.
- **FORBIDDEN**: Using plaintext send() and incrementing a counter instead of encrypting.
- **FORBIDDEN**: Disabling cert validation in tests "to make them pass."
- **FORBIDDEN**: Hardcoding the expected ciphertext in tests (capture from a real session).
- **FORBIDDEN**: Marking TLS as "production-ready" without the interop tests passing.

16 QUALITY GATES

- **Gate 1**: grep -rn 'is_connected = true\|bytes_encrypted' endc/src/ returns zero matches.
- **Gate 2**: curl interop succeeds for TLS 1.2 and 1.3.
- **Gate 3**: tcpdump capture shows ciphertext.
- **Gate 4**: All adversarial tests (expired, mismatch, MITM) reject as expected.

17 DEFINITION OF DONE

- Real TLS library integrated.
- Handshake + cert validation works.
- Encrypted read/write works.
- Interop with curl and openssl verified.
- Adversarial tests pass.
- No fake counters remain.

18 FINAL AUDIT

- **Implementation audit**: no fake TLS code remains.
- **Runtime audit**: real handshake, real encryption.
- **Security audit**: cert validation rejects invalid certs; traffic is ciphertext.
- **Evidence audit**: Final Evidence contains curl output, tcpdump capture description, adversarial test outputs.

19 FINAL EVIDENCE

- Paste grep -rn 'is_connected = true\|bytes_encrypted' endc/src/ output (zero matches).
- Paste curl -v https://localhost:8443/ output showing TLS 1.3 success.
- Paste tcpdump/Wireshark analysis showing application-data records are ciphertext.
- Paste expired-cert rejection output.

PROMPT 14

Cryptography / Argon2 / Hashing

Phase: 5 | **Priority: P1** | **Effort: 7 days** | **Depends: 02** | **Features: F-21, F-22**

1 TITLE

Cryptography / Argon2 / Hashing — replace the fake Argon2id module with the real argon2 crate, verify HMAC against RFC test vectors, and add malformed-input / error-path / interoperability tests.

2 MISSION

After execution: **(a)** endc/src/codegen/c_backend/runtime/concurrency_crypto.rs uses the real argon2 Rust crate (or emits C that links against libargon2); **(b)** password hashing uses real Argon2id with documented parameters (m=64MiB, t=3, p=4 default); **(c)** verification of an Argon2id hash against a password is constant-time; **(d)** HMAC-SHA256 is implemented using the hmac crate and verified against RFC 4231 test vectors; **(e)** SHA-256 (already using sha2) is verified against NIST FIPS 180-4 test vectors; **(f)** malformed-input tests: empty password, max-length password, hash with invalid parameters, hash with corrupted encoding — all rejected; **(g)** interop test: a hash produced by Python's argon2 library is verifiable by End, and vice versa.

3 CURRENT STATE

concurrency_crypto.rs declares Argon2-shaped C structs but does not call the real algorithm. sha2 is in Cargo.toml and used for SHA-256, but no test vectors verify correctness. No argon2, hmac crates.

4 VERIFIED PROBLEMS

- **P1**: Fake Argon2 — verified by reading the file and confirming no real algorithm.
- **P2**: No HMAC.
- **P3**: No test vectors for SHA-256.
- **P4**: No interop tests with other Argon2 implementations.

5 NON-GOALS

- Implementing custom cryptographic algorithms — the directive forbids this.
- Adding new cipher suites beyond Argon2id, SHA-256, HMAC-SHA256 (other algorithms may be added in follow-up prompts).

6 PRESERVATION REQUIREMENTS

- The End std/crypto public API shape is preserved.

7 ARCHITECTURAL REQUIREMENTS

Add argon2 = "0.x" and hmac = "0.x" to Cargo.toml. The End runtime calls into these crates via FFI (or, for emitted C, via #include <argon2.h> linking against libargon2-dev). The constant-time comparison uses subtle crate. Hash encoding follows the PHC string format.

8 IMPLEMENTATION PLAN

1. **Add** argon2, hmac, subtle crates.
2. **Implement** password hashing using Argon2id with PHC-encoded output.
3. **Implement** verification using constant-time compare.
4. **Implement** HMAC-SHA256.
5. **Write** test-vector tests for SHA-256, HMAC-SHA256, Argon2id (using RFC 9106 test vectors if available, or generate a known hash with the Python argon2 library and use that).
6. **Write** malformed-input tests.
7. **Write** interop test with Python.

9 FILE / MODULE INVESTIGATION

Inspect: endc/src/codegen/c_backend/runtime/concurrency_crypto.rs, End std/crypto/*.end files.

10 DEPENDENCY REQUIREMENTS

Add: argon2, hmac, subtle. System: optional libargon2-dev if using C path.

11 DATA / API / PROTOCOL CONTRACTS

Argon2id PHC format: `$argon2id$v=19$m=65536,t=3,p=4$<salt>$<hash>`. HMAC-SHA256 RFC 4231 test vector for key=0x0b*20, data="Hi There" produces b0344c61d8db38535ca8afceaf0bf12b881dc200c98320 8732475245255b61.

12 TEST PLAN

- **SHA-256 vectors**: empty string, "abc", long string — all match NIST vectors.
- **HMAC-SHA256 RFC 4231**: all 7 test cases match.
- **Argon2id**: hash the same password twice with the same salt — produces identical hash; hash with different salt — produces different hash; verify a correct password — returns true; verify a wrong password — returns false, in constant time.
- **Malformed**: empty password accepted (hash is still well-defined); corrupted PHC string rejected; out-of-range parameters rejected.
- **Interop**: hash password in Python, verify in End — succeeds.

13 EXECUTION TESTS

- cargo test --package endc --lib crypto — all tests pass.
- Python interop: python3 -c "import argon2; print(argon2.PasswordHasher().hash('test'))" > /tmp/py_hash; use as input to End verifier — accepts.

14 FAILURE LOOP

If a test vector fails: STOP → the implementation is wrong → fix → re-run. If interop fails: STOP → the parameter encoding differs → reconcile to the standard → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Hardcoding expected hashes in tests without computing them through the implementation.
- **FORBIDDEN**: Implementing Argon2 from scratch.
- **FORBIDDEN**: Using non-constant-time comparison for verification.

16 QUALITY GATES

- **Gate 1**: argon2, hmac, subtle in Cargo.toml.
- **Gate 2**: All RFC test vectors pass.
- **Gate 3**: Malformed-input tests reject as expected.
- **Gate 4**: Python interop succeeds.

17 DEFINITION OF DONE

- Real Argon2id implemented.
- HMAC-SHA256 implemented.
- SHA-256 verified.
- All test vectors pass.
- Malformed-input tests pass.
- Python interop passes.

18 FINAL AUDIT

- **Implementation audit**: real crypto crates used.
- **Security audit**: constant-time comparison verified.
- **Test audit**: all test vectors and malformed-input tests pass.
- **Evidence audit**: Final Evidence contains RFC test vector output and Python interop result.

19 FINAL EVIDENCE

- Paste RFC 4231 HMAC-SHA256 test vector output.
- Paste NIST SHA-256 test vector output.
- Paste Python interop test result.

PROMPT 15

GGUF / AI Runtime

Phase: 6 | **Priority: P2** | **Effort: 14 days** | **Depends: 02** | **Features: F-23**

1 TITLE

GGUF / AI Runtime — replace the file-header-reading fake GGUF module with a real candle/ort/llm integration that loads a real GGUF model file and runs inference.

2 MISSION

After execution: **(a)** endc uses candle (preferred, pure-Rust) or ort (ONNX runtime bindings) to load a real GGUF file (e.g., a tiny Llama-2 model); **(b)** inference produces real tokens — for a fixed prompt and seed, the output is deterministic and reproducible; **(c)** the inference is verified by comparing the first 10 output tokens against a known-good reference (e.g., the same model loaded via llama.cpp); **(d)** unsupported model architectures produce a clear error, not a fake inference result; **(e)** memory and time budgets are reported honestly.

3 CURRENT STATE

The GGUF module reads file headers and reports metadata but performs no inference. No candle, ort, llm crates in Cargo.toml.

4 VERIFIED PROBLEMS

- **P1**: No real inference — "GGUF supported" is false.
- **P2**: No model architecture handling.
- **P3**: No reproducibility verification.

5 NON-GOALS

- Training or fine-tuning — inference only.
- Multi-GPU inference.
- Custom kernel development.

6 PRESERVATION REQUIREMENTS

- End std/llm API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add candle-core, candle-transformers, tokenizers to Cargo.toml. The runtime loads the GGUF file, parses metadata, builds the appropriate model architecture (Llama, Mistral, etc.), and runs inference. For reproducibility, set the RNG seed and temperature to 0.

8 IMPLEMENTATION PLAN

1. **Add** candle crates.
2. **Implement** GGUF file loader using candle's existing GGUF support.
3. **Implement** inference for Llama-2 architecture.
4. **Generate** reference tokens using llama.cpp with the same model and prompt.
5. **Verify** End's inference matches the reference for the first 10 tokens.
6. **Write** adversarial tests: corrupt GGUF file rejected; unsupported architecture rejected.

9 FILE / MODULE INVESTIGATION

Inspect: the End std/llm/ module, endc/src/codegen/c_backend/runtime/ for any existing GGUF code.

10 DEPENDENCY REQUIREMENTS

Add: candle-core, candle-transformers, tokenizers. Optional: llm as alternative. System: a CPU is sufficient for tiny models; for GPU, CUDA runtime required.

11 DATA / API / PROTOCOL CONTRACTS

Inference result: `{"model":"llama-2-7b","prompt":"Once upon a time","seed":0,"temperature":0,"tokens":["Once","upon","a","time",...],"duration_ms":...}`.

12 TEST PLAN

- Reproducibility: same prompt + same seed → same first 10 tokens across runs.
- Reference match: tokens match llama.cpp output for the same configuration.
- Corrupt file rejected.
- Unsupported architecture rejected.

13 EXECUTION TESTS

- endc run examples/llm_demo.end -- --model /tmp/tinyllama.gguf --prompt "Hello" — produces real tokens.
- Compare to llama.cpp output — tokens match.

14 FAILURE LOOP

If tokens don't match the reference: STOP → check tokenizer, model architecture, sampling config → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Hardcoding the expected tokens in the test.
- **FORBIDDEN**: Returning file metadata as if it were inference output.

16 QUALITY GATES

- **Gate 1**: candle crates in Cargo.toml.
- **Gate 2**: Reproducibility test passes.
- **Gate 3**: Reference match passes for first 10 tokens.

17 DEFINITION OF DONE

- Real GGUF loading.
- Real inference.
- Reproducibility verified.
- Reference match verified.

18 FINAL AUDIT

- **Implementation audit**: real inference, not file header reading.
- **Runtime audit**: tokens produced match reference.
- **Evidence audit**: Final Evidence contains inference output and reference comparison.

19 FINAL EVIDENCE

- Paste End inference output.
- Paste llama.cpp reference output for the same prompt+seed.
- Show the comparison.

PROMPT 16

GPU Abstractions

Phase: 6 | **Priority: P2** | **Effort: 18 days** | **Depends: 02** | **Features: F-24**

1 TITLE

GPU Abstractions — replace fake GPU code with a real compute-backend integration (CUDA, Vulkan compute, or wgpu) that actually executes kernels and returns validated results.

2 MISSION

After execution: **(a)** choose one compute backend (CUDA via cudarc, or wgpu for portability); **(b)** the End std/gpu module can: allocate device memory, upload data, dispatch a kernel, download results; **(c)** a vector-add kernel produces correct results for input sizes 1k, 1M, 64M; **(d)** a matrix-multiply kernel produces correct results verifiable against a CPU reference; **(e)** failure modes are real: no GPU available → clear error; kernel compilation error → reported; out-of-memory → reported.

3 CURRENT STATE

The GPU module emits C structs with the right names but no kernel execution. No cudarc, wgpu, vulkano in Cargo.toml.

4 VERIFIED PROBLEMS

- **P1**: No real GPU execution.
- **P2**: No CUDA/wgpu/Vulkan integration.

5 NON-GOALS

- Multi-backend support in v1 — pick one.
- Custom kernel language — use the chosen backend's existing kernel format (PTX for CUDA, WGSL for wgpu).

6 PRESERVATION REQUIREMENTS

- End std/gpu API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add wgpu = "0.x" (preferred for portability). Implement: GpuBuffer, GpuKernel, dispatch(). Kernels are written in WGSL. The runtime compiles the WGSL, allocates buffers, dispatches, and reads back.

8 IMPLEMENTATION PLAN

1. **Add** wgpu crate.
2. **Implement** device initialization with adapter selection.
3. **Implement** buffer allocation and upload/download.
4. **Implement** kernel compilation from WGSL.
5. **Implement** dispatch.
6. **Write** vector-add and matrix-multiply tests with CPU reference comparison.

9 FILE / MODULE INVESTIGATION

Inspect: the existing fake GPU module, End std/gpu/ files.

10 DEPENDENCY REQUIREMENTS

Add wgpu = "0.x". System: Vulkan loader (libvulkan1) on Linux.

11 DATA / API / PROTOCOL CONTRACTS

GPU dispatch evidence: `{"adapter":"NVIDIA GeForce RTX 4090","workgroup_size":[64,1,1],"dispatch_size":[16384,1,1],"duration_ms":12,"output_sha256":"..."}`.

12 TEST PLAN

- Vector-add: 1k, 1M, 64M elements — all match CPU reference.
- Matrix-multiply: 256x256 — matches CPU reference within floating-point tolerance.
- No-GPU: on a system without GPU, the runtime reports a clear error.

13 EXECUTION TESTS

- endc run examples/gpu_vec_add.end — produces correct sum.
- Compare to CPU reference output.

14 FAILURE LOOP

If GPU output mismatches CPU reference: STOP → check kernel, buffer sizes, dispatch dims → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Hardcoding expected GPU output.
- **FORBIDDEN**: Falling back to CPU silently when GPU is requested.

16 QUALITY GATES

- **Gate 1**: wgpu in Cargo.toml.
- **Gate 2**: Vector-add tests pass for all sizes.
- **Gate 3**: Matrix-multiply test passes.

17 DEFINITION OF DONE

- Real GPU integration.
- Real kernel execution.
- Tests pass against CPU reference.

18 FINAL AUDIT

- **Implementation audit**: real wgpu integration.
- **Runtime audit**: kernels actually execute.
- **Evidence audit**: output matches CPU reference.

19 FINAL EVIDENCE

- Paste vector-add test output.
- Paste matrix-multiply test output.

PROMPT 17 SQLite Compatibility (Real Engine)

Phase: 5 | **Priority: P1** | **Effort: 8 days** | **Depends: 02** | **Features: F-25**

1 TITLE

SQLite Compatibility (Real Engine) — replace the key-value text file fake-SQLite with a real rusqlite integration that creates real .db files readable by the official sqlite3 CLI.

2 MISSION

After execution: **(a)** End std/sqlite uses rusqlite (bundling SQLite via libsqlite3-sys); **(b)** a database created by End can be opened and queried by the official sqlite3 CLI; **(c)** SQL queries (CREATE TABLE, INSERT, SELECT with WHERE/JOIN, UPDATE, DELETE) work correctly; **(d)** transactions are atomic; **(e)** concurrency test: two processes writing simultaneously — one blocks until the other commits; **(f)** prepared statements work; **(g)** the fake key-value text-file module is removed.

3 CURRENT STATE

The SQLite module writes key-value pairs to a text file with .db extension. sqlite3 file.db reports "file is not a database." No rusqlite in Cargo.toml.

4 VERIFIED PROBLEMS

- **P1**: Text-file impostor.
- **P2**: No real SQL execution.
- **P3**: "SQLite compatible" claim is a documentation lie.

5 NON-GOALS

- Custom query planner — use SQLite's.
- Custom SQL dialect — use SQLite's SQL.

6 PRESERVATION REQUIREMENTS

- End std/sqlite API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add rusqlite = { version = "0.x", features = ["bundled"] } to Cargo.toml. The bundled feature compiles SQLite from source, no system dependency. The End runtime exposes Connection::open, execute, prepare, query_map, transaction methods.

8 IMPLEMENTATION PLAN

1. **Add** rusqlite with bundled feature.
2. **Implement** connection, execute, prepare, query.
3. **Implement** transactions.
4. **Write** tests: CRUD, JOIN, transaction rollback, prepared statement reuse.
5. **Write** interop test: create DB in End, open in sqlite3 CLI, run SELECT * FROM t; — succeeds.
6. **Write** concurrency test: two processes, one blocks.

9 FILE / MODULE INVESTIGATION

Inspect: the fake SQLite module, End std/sqlite/ files.

10 DEPENDENCY REQUIREMENTS

Add rusqlite = { version = "0.x", features = ["bundled"] }.

11 DATA / API / PROTOCOL CONTRACTS

SQLite file format: the official SQLite file format. Interop: sqlite3 end_created.db "SELECT * FROM users;" returns rows.

12 TEST PLAN

- **CRUD**: CREATE TABLE, INSERT, SELECT, UPDATE, DELETE.
- **JOIN**: two tables, JOIN, expected results.
- **Transaction**: BEGIN, INSERT, ROLLBACK, SELECT — no row.
- **Prepared statement**: same statement, multiple parameter sets.
- **Interop**: create in End, open in sqlite3 CLI — success.
- **Concurrency**: two processes write simultaneously — serialized.

13 EXECUTION TESTS

- endc run examples/sqlite_demo.end — creates /tmp/test.db.
- sqlite3 /tmp/test.db "SELECT * FROM users;" — returns rows.

14 FAILURE LOOP

If sqlite3 CLI cannot open the file: STOP → the file is not real SQLite → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Using the text-file module as a fallback.
- **FORBIDDEN**: Hardcoding SELECT results in tests.

16 QUALITY GATES

- **Gate 1**: rusqlite in Cargo.toml.
- **Gate 2**: CRUD tests pass.
- **Gate 3**: sqlite3 CLI interop succeeds.
- **Gate 4**: Concurrency test passes.

17 DEFINITION OF DONE

- Real rusqlite integration.
- Real SQL execution.
- Real SQLite file format.
- Interop with sqlite3 CLI.
- Concurrency works.

18 FINAL AUDIT

- **Implementation audit**: real rusqlite.
- **Runtime audit**: real SQL.
- **Interop audit**: sqlite3 CLI reads the file.
- **Evidence audit**: Final Evidence contains sqlite3 CLI output.

19 FINAL EVIDENCE

- Paste sqlite3 /tmp/test.db ".tables" output.
- Paste sqlite3 /tmp/test.db "SELECT * FROM users;" output.
- Paste concurrency test output.

PROMPT 18

PostgreSQL Wire Protocol

Phase: 5 | **Priority: P1** | **Effort: 10 days** | **Depends: 02** | **Features: F-26**

1 TITLE

PostgreSQL Wire Protocol — replace the fake PostgreSQL module with a real tokio-postgres (or postgres synchronous) integration that connects to a real PostgreSQL server, performs the wire handshake, executes queries, and returns real result sets.

2 MISSION

After execution: **(a)** End std/postgres uses tokio-postgres; **(b)** connect to a real postgres server (run in Docker for tests); **(c)** execute SELECT 1, SELECT * FROM users WHERE id=$1, INSERT INTO users(name) VALUES($1); **(d)** prepared statements work with parameter binding; **(e)** transactions work; **(f)** NULL values are correctly handled; **(g)** interop test: connect to a real PostgreSQL 16 server, perform CRUD, verify data via psql CLI.

3 CURRENT STATE

Fake PostgreSQL module emits structs with the right names but does not perform the wire handshake. No tokio-postgres/postgres in Cargo.toml.

4 VERIFIED PROBLEMS

- **P1**: No real wire handshake.
- **P2**: No real query execution.
- **P3**: "PostgreSQL compatible" is false.

5 NON-GOALS

- Implementing the wire protocol from scratch — use the library.
- Connection pooling (PgBator etc.) — out of scope.

6 PRESERVATION REQUIREMENTS

- End std/postgres API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add tokio = { version = "1", features = ["full"] } and tokio-postgres = "0.x". The End runtime's async runtime uses tokio. The postgres module exposes connect(conn_str), execute(sql, params), query(sql, params), transaction methods.

8 IMPLEMENTATION PLAN

1. **Add** tokio + tokio-postgres crates.
2. **Set up** a PostgreSQL test server via Docker (docker run -d -p 5432:5432 postgres:16).
3. **Implement** connect, execute, query, transaction.
4. **Write** tests: CRUD, JOIN, transaction rollback, NULL handling.
5. **Write** interop: insert via End, verify via psql.

9 FILE / MODULE INVESTIGATION

Inspect: the fake PostgreSQL module, End std/postgres/ files.

10 DEPENDENCY REQUIREMENTS

Add: tokio, tokio-postgres. System: PostgreSQL server for tests (Docker).

11 DATA / API / PROTOCOL CONTRACTS

Connection string format: postgresql://user:pass@localhost:5432/dbname. Query result: {"rows":[{"id":1,"name":"alice"}],"rows_affected":0}.

12 TEST PLAN

- CRUD: CREATE TABLE, INSERT, SELECT, UPDATE, DELETE.
- JOIN: two-table SELECT.
- Parameter binding: WHERE id=$1 with various types.
- Transaction: BEGIN, INSERT, ROLLBACK, SELECT — no row.
- NULL: insert NULL, select NULL — handled correctly.
- Interop: insert via End, query via psql -c "SELECT * FROM users" — shows the row.

13 EXECUTION TESTS

- Start docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=test postgres:16.
- endc run examples/postgres_demo.end — performs CRUD.
- psql -h localhost -U postgres -d test -c "SELECT * FROM users" — shows rows inserted by End.

14 FAILURE LOOP

If psql cannot see End's data: STOP → transaction not committed → fix → re-run. If connection fails: STOP → check credentials, network → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Hardcoding SELECT results.
- **FORBIDDEN**: Using SQLite as a "PostgreSQL-compatible" fallback.

16 QUALITY GATES

- **Gate 1**: tokio-postgres in Cargo.toml.
- **Gate 2**: CRUD tests pass against real PostgreSQL.
- **Gate 3**: psql interop succeeds.
- **Gate 4**: NULL handling test passes.

17 DEFINITION OF DONE

- Real PostgreSQL integration.
- Real wire handshake.
- Real query execution.
- Interop with psql verified.

18 FINAL AUDIT

- **Implementation audit**: real library.
- **Runtime audit**: real queries.
- **Interop audit**: psql confirms.

19 FINAL EVIDENCE

- Paste psql -c "SELECT * FROM users" output.
- Paste End test output.

PROMPT 19

Redis Wire Protocol

Phase: 5 | **Priority: P2** | **Effort: 6 days** | **Depends: 02** | **Features: F-27**

1 TITLE

Redis Wire Protocol — replace the fake Redis module with a real redis crate integration that connects to a real Redis server and executes commands.

2 MISSION

After execution: **(a)** End std/redis uses redis crate; **(b)** connect to real Redis (Docker); **(c)** SET, GET, DEL, EXPIRE, HSET/HGET, LPUSH/RPOP all work; **(d)** pipelining works; **(e)** pub/sub works; **(f)** interop: set a key in End, verify via redis-cli GET key.

3 CURRENT STATE

Fake Redis module. No redis crate.

4 VERIFIED PROBLEMS

- **P1**: No RESP protocol.
- **P2**: No real command execution.

5 NON-GOALS

- Cluster mode — single-node only in v1.
- Custom commands — use the standard command set.

6 PRESERVATION REQUIREMENTS

- End std/redis API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add redis = "0.x" to Cargo.toml.

8 IMPLEMENTATION PLAN

1. **Add** redis crate.
2. **Docker** Redis test server.
3. **Implement** commands.
4. **Tests**: strings, hashes, lists, pipelining, pub/sub.

9 FILE / MODULE INVESTIGATION

Inspect: the fake Redis module.

10 DEPENDENCY REQUIREMENTS

Add: redis = "0.x". System: Redis (Docker).

11 DATA / API / PROTOCOL CONTRACTS

RESP protocol per Redis specification. Command API: set(key, value), get(key), del(key), etc.

12 TEST PLAN

- String SET/GET round-trips correctly.
- Hash HSET/HGET.
- List LPUSH/RPOP.
- Pipelining: 100 commands in one round-trip.
- Pub/sub: subscribe, publish, receive.
- Interop: SET in End, GET via redis-cli.

13 EXECUTION TESTS

- docker run -d -p 6379:6379 redis:7.
- endc run examples/redis_demo.end — performs SET.
- redis-cli GET mykey — returns the value set by End.

14 FAILURE LOOP

If redis-cli cannot see the value: STOP → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Hardcoding GET results.
- **FORBIDDEN**: In-memory fake-Redis fallback.

16 QUALITY GATES

- **Gate 1**: redis in Cargo.toml.
- **Gate 2**: All command tests pass.
- **Gate 3**: redis-cli interop succeeds.

17 DEFINITION OF DONE

- Real redis integration.
- All commands work.
- Pub/sub works.
- Interop verified.

18 FINAL AUDIT

- **Implementation audit**: real library.
- **Runtime audit**: real commands.
- **Interop audit**: redis-cli confirms.

19 FINAL EVIDENCE

- Paste redis-cli GET mykey output.
- Paste pub/sub test output.

PROMPT 20

HTTP/2 + HPACK

Phase: 5 | **Priority: P2** | **Effort: 8 days** | **Depends: 02, 13** | **Features: F-28**

1 TITLE

HTTP/2 + HPACK — replace the fake HTTP/2 module with a real h2 crate integration that performs the protocol handshake, frame parsing, and HPACK header compression against a real HTTP/2 server.

2 MISSION

After execution: **(a)** End std/http2 uses h2 crate; **(b)** HTTP/2 connection to a real HTTP/2 server (e.g., nghttp2-based server or nginx with HTTP/2 enabled); **(c)** multiplexed streams work; **(d)** HPACK-encoded headers are correctly decoded; **(e)** interop: curl --http2 https://localhost:8443/ succeeds against an End HTTP/2 server.

3 CURRENT STATE

Fake HTTP/2 module. No h2 in Cargo.toml.

4 VERIFIED PROBLEMS

- **P1**: No real HTTP/2.
- **P2**: No HPACK.

5 NON-GOALS

- HTTP/3 / QUIC — out of scope.
- gRPC — out of scope.

6 PRESERVATION REQUIREMENTS

- End std/http2 API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add h2 = "0.x". Use the existing TLS layer from Prompt 13 for ALPN negotiation.

8 IMPLEMENTATION PLAN

1. **Add** h2 crate.
2. **Implement** client and server.
3. **Tests**: multiplexed streams, HPACK decode, header compression ratio.
4. **Interop**: curl --http2.

9 FILE / MODULE INVESTIGATION

Inspect: the fake HTTP/2 module.

10 DEPENDENCY REQUIREMENTS

Add: h2 = "0.x".

11 DATA / API / PROTOCOL CONTRACTS

HTTP/2 per RFC 7540. HPACK per RFC 7541.

12 TEST PLAN

- Multiplexing: 10 concurrent streams over one connection.
- HPACK: header set encoded and decoded correctly.
- Interop: curl --http2 https://localhost:8443/ succeeds.

13 EXECUTION TESTS

- endc run examples/http2_server.end — starts HTTP/2 server.
- curl --http2 -v https://localhost:8443/ — succeeds.

14 FAILURE LOOP

If curl fails: STOP → reproduce with nghttp -v → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Falling back to HTTP/1.1 when HTTP/2 is requested.

16 QUALITY GATES

- **Gate 1**: h2 in Cargo.toml.
- **Gate 2**: Multiplexing test passes.
- **Gate 3**: curl --http2 interop succeeds.

17 DEFINITION OF DONE

- Real HTTP/2 + HPACK.
- Multiplexing works.
- Interop verified.

18 FINAL AUDIT

- **Implementation audit**: real library.
- **Runtime audit**: real protocol.
- **Interop audit**: curl confirms.

19 FINAL EVIDENCE

- Paste curl --http2 -v output.
- Paste multiplexing test output.

PROMPT 21

Atomics / Mutex (Concurrency Primitives)

Phase: 5 | **Priority: P1** | **Effort: 6 days** | **Depends: 02, 05** | **Features: F-29**

1 TITLE

Atomics / Mutex — audit and verify the emitted C11 <stdatomic.h> and pthread mutex code for correctness; add adversarial concurrency tests that would expose data races, deadlocks, and incorrect ordering.

2 MISSION

After execution: **(a)** every emitted atomic operation maps to the correct C11 atomic_* function; **(b)** every emitted mutex operation maps to correct pthread (or C11 mtx_*) calls; **(c)** memory orderings (Relaxed, Acquire, Release, AcqRel, SeqCst) are correctly translated; **(d)** adversarial tests: 10 producer-consumer, 10 readers-writers, 5 deadlocking scenarios (verified to NOT deadlock under correct usage, verified to deadlock under incorrect usage with a known-bad pattern); **(e)** ThreadSanitizer (TSan) run on the test programs reports zero data races for correct programs; **(f)** TSan reports races for deliberately-incorrect programs.

3 CURRENT STATE

The atomics module emits C11 <stdatomic.h>-shaped C but the correctness is unverified. No TSan tests.

4 VERIFIED PROBLEMS

- **P1**: Correctness unverified.
- **P2**: No memory ordering tests.
- **P3**: No TSan verification.

5 NON-GOALS

- Custom async runtime — use threads.
- Actor model — out of scope.

6 PRESERVATION REQUIREMENTS

- End std/sync API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add tsan compilation flag support. The runner can compile generated C with gcc -fsanitize=thread for TSan mode. Add an option --tsan to endc build.

8 IMPLEMENTATION PLAN

1. **Inventory** every emitted atomic and mutex operation.
2. **Verify** mapping to C11/pthread is correct.
3. **Add** memory ordering tests.
4. **Compile** tests with TSan.
5. **Write** adversarial concurrency tests.

9 FILE / MODULE INVESTIGATION

Inspect: the atomics module, End std/sync/ files.

10 DEPENDENCY REQUIREMENTS

System: GCC with TSan support (already available).

11 DATA / API / PROTOCOL CONTRACTS

TSan output: WARNING: ThreadSanitizer: data race ... on incorrect programs; clean output on correct programs.

12 TEST PLAN

- Producer-consumer: 10 producers, 10 consumers, 10k items each — final count correct, TSan clean.
- Readers-writers: 10 readers, 2 writers — TSan clean.
- Deliberate race: two threads writing same variable without atomic — TSan reports.
- Deadlock: known-bad lock ordering — test times out.

13 EXECUTION TESTS

- endc build --tsan examples/threads_demo.end && ./threads_demo — runs, TSan clean.
- Deliberate-race program: ./race_demo — TSan reports race.

14 FAILURE LOOP

If TSan reports a race in a "correct" program: STOP → the codegen is wrong → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Disabling TSan in tests.
- **FORBIDDEN**: Marking race-detection tests as "flaky" and ignoring.

16 QUALITY GATES

- **Gate 1**: All correct-program tests are TSan-clean.
- **Gate 2**: All deliberate-race tests produce TSan reports.

17 DEFINITION OF DONE

- Atomic ops verified.
- Memory orderings verified.
- TSan integration works.
- Adversarial tests pass.

18 FINAL AUDIT

- **Implementation audit**: correct C11/pthread mapping.
- **Runtime audit**: TSan clean on correct, reports on incorrect.

19 FINAL EVIDENCE

- Paste TSan clean output for producer-consumer.
- Paste TSan race report for deliberate-race program.

PROMPT 22

Raft Consensus (Real Implementation)

Phase: 6 | **Priority: P2** | **Effort: 15 days** | **Depends: 02, 17, 21** | **Features: F-30**

1 TITLE

Raft Consensus — replace the fake Raft module with a real implementation (or integration of openraft/raft crate) that performs leader election, log replication, and commit on a 3-node cluster.

2 MISSION

After execution: **(a)** End std/raft uses openraft = "0.x" or implements Raft directly; **(b)** a 3-node cluster can be started on localhost; **(c)** leader election completes within 5 seconds of cluster start; **(d)** a write to the leader is replicated to all followers within 1 second; **(e)** leader failure (kill the leader process) triggers re-election within 10 seconds; **(f)** log entries are persisted (using the real SQLite from Prompt 17 as the persistent log); **(g)** the cluster survives network partition (minority side rejects writes, majority side continues).

3 CURRENT STATE

Fake Raft module — structurally shaped but no leader election, no log replication. No openraft or raft in Cargo.toml.

4 VERIFIED PROBLEMS

- **P1**: No leader election.
- **P2**: No log replication.
- **P3**: No persistence.

5 NON-GOALS

- Custom Raft variant — use the standard protocol.
- Multi-Raft / sharding — out of scope.
- Dynamic membership change — out of scope.

6 PRESERVATION REQUIREMENTS

- End std/raft API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add openraft = "0.x" (or raft = "0.x"). The runtime exposes RaftNode::start(cluster_config), write(entry), read(). The persistent log uses SQLite via Prompt 17.

8 IMPLEMENTATION PLAN

1. **Add** openraft crate.
2. **Implement** cluster startup.
3. **Implement** leader election observation.
4. **Implement** write/read.
5. **Implement** leader-failure test.
6. **Implement** network partition test (using iptables or tc).
7. **Use** SQLite as the persistent log store.

9 FILE / MODULE INVESTIGATION

Inspect: the fake Raft module.

10 DEPENDENCY REQUIREMENTS

Add: openraft = "0.x", tokio (already from Prompt 18).

11 DATA / API / PROTOCOL CONTRACTS

Raft per *In Search of an Understandable Consensus Algorithm*. Term numbers, log indices, commit index all per the paper.

12 TEST PLAN

- 3-node startup, leader elected in < 5s.
- Write 100 entries; verify all 3 nodes have them.
- Kill leader; new leader elected in < 10s.
- Restart killed node; it catches up.
- Network partition (2 nodes isolated): minority rejects writes, majority continues.

13 EXECUTION TESTS

- Start 3 End processes with cluster config.
- Write to leader, read from any node — data is consistent.
- Kill -9 the leader process — cluster re-elects.

14 FAILURE LOOP

If leader election fails: STOP → reproduce in isolation → check transport, timing → fix → re-run. If partition test fails: STOP → check majority handling → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Hardcoding "leader elected" without actual election.
- **FORBIDDEN**: Marking partition test as "manual" to skip it.

16 QUALITY GATES

- **Gate 1**: openraft in Cargo.toml.
- **Gate 2**: Leader election test passes.
- **Gate 3**: Write replication test passes.
- **Gate 4**: Leader-failure re-election test passes.
- **Gate 5**: Partition test passes.

17 DEFINITION OF DONE

- Real Raft integration.
- Leader election works.
- Replication works.
- Failure recovery works.
- Partition handling works.

18 FINAL AUDIT

- **Implementation audit**: real library.
- **Runtime audit**: cluster operates correctly.
- **Failure audit**: recovery works.

19 FINAL EVIDENCE

- Paste leader election log.
- Paste write replication test output (all 3 nodes have the same entries).
- Paste leader-failure re-election log.

PROMPT 23

Profiler (Real Measurement)

Phase: 6 | **Priority: P2** | **Effort: 6 days** | **Depends: 02** | **Features: F-31**

1 TITLE

Profiler (Real Measurement) — replace the constant-returning fake profiler with a real measurement integration using perf (Linux), gperftools, or pprof-rs that produces honest timing and call-graph data.

2 MISSION

After execution: **(a)** the End profiler wraps pprof-rs (in-process) or shells out to perf (out-of-process); **(b)** profiling two programs with measurably different behavior (one CPU-bound tight loop, one I/O-bound sleep loop) produces measurably different profiles — the CPU-bound program shows 99% time in the loop, the I/O-bound shows 99% in nanosleep; **(c)** flame-graph output is generated; **(d)** no hardcoded timing values; **(e)** the profiler output is reproducible within a documented variance.

3 CURRENT STATE

The profiler returns constant timing and call-count values regardless of program behavior. No pprof-rs, no perf integration.

4 VERIFIED PROBLEMS

- **P1**: Constants returned as measurements.
- **P2**: No real profiling tool integration.

5 NON-GOALS

- Custom profiler implementation — use established tools.
- Continuous profiling in production — out of scope.

6 PRESERVATION REQUIREMENTS

- End std/profile API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add pprof = "0.x" to Cargo.toml. The runtime exposes Profiler::start(duration), stop() -> ProfileReport. The report contains: per-function call counts, per-function CPU time, flame-graph SVG path. For the perf path, the runtime spawns perf record -p <pid>, waits, stops, runs perf script, parses the output.

8 IMPLEMENTATION PLAN

1. **Add** pprof-rs.
2. **Implement** start/stop.
3. **Implement** flame-graph output (use inferno crate for SVG).
4. **Write** differential test: CPU-bound vs I/O-bound programs produce measurably different profiles.
5. **Write** reproducibility test: profile same program twice, variance < 20%.

9 FILE / MODULE INVESTIGATION

Inspect: the fake profiler module.

10 DEPENDENCY REQUIREMENTS

Add: pprof = "0.x", inferno. System: Linux for perf path.

11 DATA / API / PROTOCOL CONTRACTS

Profile report JSON: `{"duration_ms":5000,"samples":5000,"functions":[{"name":"fib","samples":4500,"percent":90.0},{"name":"main","samples":500,"percent":10.0}],"flame_graph_svg":"..."}`.

12 TEST PLAN

- CPU-bound: fib(40) in tight loop — profile shows 99% in fib.
- I/O-bound: 5 seconds of sleep — profile shows 99% in nanosleep.
- Reproducibility: same program profiled twice, variance < 20%.
- Flame-graph SVG is generated and openable in a browser.

13 EXECUTION TESTS

- endc profile --duration 5s examples/fib_loop.end — produces report with fib as top function.
- endc profile --duration 5s examples/sleep_loop.end — produces report with nanosleep as top function.

14 FAILURE LOOP

If both profiles look identical: STOP → the profiler is returning constants → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Hardcoding function names or sample counts in profiler output.
- **FORBIDDEN**: Marking the differential test as "flaky" to skip it.

16 QUALITY GATES

- **Gate 1**: pprof in Cargo.toml.
- **Gate 2**: Differential test passes (profiles differ).
- **Gate 3**: Reproducibility test passes (variance < 20%).

17 DEFINITION OF DONE

- Real profiler integration.
- Differential test passes.
- Reproducibility passes.
- Flame-graph output works.

18 FINAL AUDIT

- **Implementation audit**: real library.
- **Runtime audit**: profiles reflect actual behavior.

19 FINAL EVIDENCE

- Paste profile report JSON for CPU-bound program.
- Paste profile report JSON for I/O-bound program (must differ).

PROMPT 24

Simulate / Stress Infrastructure

Phase: 6 | **Priority: P2** | **Effort: 5 days** | **Depends: 02** | **Features: F-32**

1 TITLE

Simulate / Stress Infrastructure — replace the random-number-returning fake stress/simulate module with a real load generator using wrk, hey, or a custom Rust load generator that drives real HTTP/SQL/Redis traffic against the system under test.

2 MISSION

After execution: **(a)** the End simulate module wraps a real load generator (preferred: in-process Rust using tokio + reqwest); **(b)** the generator sends real HTTP requests to a test server and measures response time, throughput, error rate; **(c)** two test scenarios with measurably different performance (fast endpoint < 1ms vs slow endpoint 50ms) produce measurably different reports; **(d)** no random numbers returned as measurements; **(e)** the report includes p50, p90, p99, p99.9 latency, throughput, error rate.

3 CURRENT STATE

The simulate module returns random numbers as latency and throughput. No real load generation.

4 VERIFIED PROBLEMS

- **P1**: Random numbers as measurements.
- **P2**: No real traffic generated.

5 NON-GOALS

- Distributed load testing — single-node load gen is sufficient.
- Custom protocol testing — HTTP only in v1.

6 PRESERVATION REQUIREMENTS

- End std/stress API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Add reqwest = "0.x", tokio. The runtime exposes StressRunner::new(target_url), .concurrency(n), .duration(d), .run() -> Report. The report contains: histogram of latencies (HdrHistogram via hdrhistogram crate), throughput, error rate, status code distribution.

8 IMPLEMENTATION PLAN

1. **Add** reqwest, hdrhistogram.
2. **Implement** load generator.
3. **Implement** latency histogram and percentile computation.
4. **Stand up** a test HTTP server with a fast endpoint (200 OK immediately) and a slow endpoint (50ms delay).
5. **Write** differential test: fast endpoint p99 < 5ms; slow endpoint p99 > 45ms.

9 FILE / MODULE INVESTIGATION

Inspect: the fake simulate module.

10 DEPENDENCY REQUIREMENTS

Add: reqwest, hdrhistogram.

11 DATA / API / PROTOCOL CONTRACTS

Stress report JSON: `{"duration_s":10,"concurrency":50,"total_requests":50000,"successful":49998,"errors":2,"latency":{"p50":1.2,"p90":2.1,"p99":5.4,"p99.9":12.3},"throughput_rps":5000}`.

12 TEST PLAN

- Fast endpoint test: p99 < 5ms.
- Slow endpoint test: p99 > 45ms.
- Error test: stress an endpoint that returns 500 — error rate > 50%.
- Reproducibility: same test twice, throughput within 10%.

13 EXECUTION TESTS

- Start a test HTTP server.
- endc stress --url http://localhost:8080/fast --duration 10s --concurrency 50 — produces report with low p99.
- endc stress --url http://localhost:8080/slow --duration 10s --concurrency 50 — produces report with high p99.

14 FAILURE LOOP

If both reports look identical: STOP → the runner is returning constants → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Hardcoding latency or throughput values.
- **FORBIDDEN**: Skipping the differential test.

16 QUALITY GATES

- **Gate 1**: reqwest, hdrhistogram in Cargo.toml.
- **Gate 2**: Differential test passes.
- **Gate 3**: Reproducibility test passes.

17 DEFINITION OF DONE

- Real load generator.
- Differential test passes.
- Real latency percentiles computed.

18 FINAL AUDIT

- **Implementation audit**: real library.
- **Runtime audit**: real traffic.

19 FINAL EVIDENCE

- Paste stress report for fast endpoint.
- Paste stress report for slow endpoint (must differ).

PROMPT 25

Attestation / Security Boundary

Phase: 6 | **Priority: P2** | **Effort: 12 days** | **Depends: 02** | **Features: F-33**

1 TITLE

Attestation / Security Boundary — replace the fake attestation module with a real integration against a hardware-backed attestation service or, on systems without TPM/SGX, an honest "software attestation" that signs a system-state hash with a project-local key.

2 MISSION

After execution: **(a)** the End attestation module produces a real signed quote of system state (binary hash, environment variables, file hashes); **(b)** the signature is verifiable by a third-party tool; **(c)** on systems with TPM 2.0, the quote uses the TPM's signing key; on systems without, the quote uses a software key with explicit attestation_kind: "software" marker; **(d)** no fake "verified" status — the verifier either gets a real quote or an explicit "no TPM available" error; **(e)** adversarial test: tamper with the binary after signing — verifier rejects.

3 CURRENT STATE

Fake attestation module returns "attested: true" without any real measurement.

4 VERIFIED PROBLEMS

- **P1**: Fake attestation.
- **P2**: No TPM/SGX integration.
- **P3**: No measurement.

5 NON-GOALS

- Full SGX enclave support — out of scope for v1.
- Confidential computing — out of scope.

6 PRESERVATION REQUIREMENTS

- End std/attest API shape preserved.

7 ARCHITECTURAL REQUIREMENTS

Use tpm-rs or shell out to tpm2-tools for TPM quotes. On systems without TPM, fall back to software attestation: sign the binary sha256 + env-var hash + dependency-file hashes with the project HMAC key (from Prompt 08). The quote includes attestation_kind field set to "tpm2" or "software". The verifier never reports "attested: true" without a real quote.

8 IMPLEMENTATION PLAN

1. **Detect** TPM availability.
2. **Implement** TPM quote via tpm2-tools subprocess.
3. **Implement** software fallback.
4. **Implement** verifier.
5. **Write** adversarial test: tamper binary, verifier rejects.

9 FILE / MODULE INVESTIGATION

Inspect: the fake attestation module.

10 DEPENDENCY REQUIREMENTS

Optional: tpm-rs. System: tpm2-tools for TPM path.

11 DATA / API / PROTOCOL CONTRACTS

Quote JSON: `{"kind":"software","binary_sha256":"...","env_hash":"...","dependency_hashes":{...},"signature":"hmac-sha256:...","timestamp":"..."}`. TPM quote: PCR values + TPM signature.

12 TEST PLAN

- Software attestation: produce quote, verify, tamper binary, verifier rejects.
- TPM path (if TPM available): produce quote, verify with tpm2_checkquote.
- No-TPM path: explicit error if TPM is requested but unavailable.

13 EXECUTION TESTS

- endc attest --binary ./hello — produces a quote.
- Tamper the binary; rerun verifier — rejects.

14 FAILURE LOOP

If tampered binary is accepted: STOP → the verifier is not actually checking → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Hardcoding "attested: true".
- **FORBIDDEN**: Marking software attestation as "tpm2".

16 QUALITY GATES

- **Gate 1**: Real attestation works (software or TPM).
- **Gate 2**: Tamper test rejects.

17 DEFINITION OF DONE

- Real attestation produced.
- Tamper detection works.
- Honest kind labeling.

18 FINAL AUDIT

- **Implementation audit**: real measurement.
- **Security audit**: tamper detection.

19 FINAL EVIDENCE

- Paste quote JSON.
- Paste tamper-test rejection output.

PROMPT 26

Standard Library Recovery

Phase: 7 | **Priority: P0** | **Effort: 14 days** | **Depends: 02, 05, 13, 14, 17, 18, 19**

Features: F-34, F-40

1 TITLE

Standard Library Recovery — audit every std/ module, classify as REAL/PARTIAL/FAKE/FACADE, replace facades with real implementations (or honestly mark them as EXPERIMENTAL), and add integration tests for every module claiming interoperability.

2 MISSION

After execution: **(a)** every std/*.end file is classified in endc/tests/stdlib_matrix.yaml; **(b)** facades are either replaced with real implementations (when the underlying subsystem is real) or marked EXPERIMENTAL in their header comment; **(c)** no std/ module claims production status without an integration test; **(d)** the HTTP/1.x server module (F-40) is brought to REAL status with conformance tests (using httpercheck or a similar HTTP conformance suite); **(e)** the matrix runner exits non-zero if any stdlib module marked REAL lacks an integration test.

3 CURRENT STATE

std/ contains ~280 .end files. Many are facades that call into fake runtime modules. After Prompts 13, 14, 17, 18, 19, the underlying TLS, crypto, SQLite, PostgreSQL, and Redis modules are real; the stdlib facades that call into them can now be made real.

4 VERIFIED PROBLEMS

- **P1**: Many stdlib modules are facades.
- **P2**: No integration tests for stdlib.
- **P3**: HTTP/1.x server lacks conformance tests.

5 NON-GOALS

- Adding new stdlib modules — only recover existing ones.
- Implementing modules whose underlying subsystem is still fake (e.g., Raft — defer until Prompt 22).

6 PRESERVATION REQUIREMENTS

- Existing API shapes preserved.

7 ARCHITECTURAL REQUIREMENTS

The matrix YAML lists every std/*.end file with: name, status (REAL/PARTIAL/FAKE/EXPERIMENTAL), depends_on (other modules), integration_test (path or null). The runner reads the matrix, runs all listed integration tests, and exits non-zero on any REAL module without a passing test.

8 IMPLEMENTATION PLAN

1. **Walk** std/ and produce the matrix YAML.
2. **For each facade**, decide: real implementation (if subsystem is real) or EXPERIMENTAL marker.
3. **Write** integration tests for every REAL module.
4. **Bring HTTP/1.x to REAL** via conformance tests.
5. **Wire** matrix runner into CI.

9 FILE / MODULE INVESTIGATION

Inspect: std/ directory tree.

10 DEPENDENCY REQUIREMENTS

Depends on subsystem crates added by previous prompts.

11 DATA / API / PROTOCOL CONTRACTS

Matrix YAML: `modules:\n - name: std/http\n status: REAL\n integration_test: tests/stdlib/http_integration.end`.

12 TEST PLAN

- Every REAL module has a passing integration test.
- Every EXPERIMENTAL module has a header comment marking it as such.
- HTTP/1.x conformance tests pass (RFC 7230 message format, RFC 7231 method semantics, RFC 7232 conditional requests, RFC 7233 range requests).

13 EXECUTION TESTS

- cargo run --bin stdlib_matrix_runner -- --coverage — prints per-module coverage.
- HTTP conformance suite passes.

14 FAILURE LOOP

If a REAL module's integration test fails: STOP → either the module is broken or the underlying subsystem is broken → fix at the right layer → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Marking a facade as REAL without an integration test.
- **FORBIDDEN**: Silently leaving a facade as REAL without replacing its internals.

16 QUALITY GATES

- **Gate 1**: Matrix YAML exists and is complete.
- **Gate 2**: All REAL modules have passing integration tests.
- **Gate 3**: HTTP conformance suite passes.

17 DEFINITION OF DONE

- Matrix complete.
- All facades resolved (real or EXPERIMENTAL).
- HTTP/1.x REAL with conformance tests.

18 FINAL AUDIT

- **Implementation audit**: matrix accurate.
- **Test audit**: all REAL modules tested.

19 FINAL EVIDENCE

- Paste matrix coverage output.
- Paste HTTP conformance suite output.

PROMPT 27

Benchmark Methodology & Integrity

Phase: 8 | **Priority: P1** | **Effort: 6 days** | **Depends: 02, 05** | **Features: F-35**

1 TITLE

Benchmark Methodology & Integrity — fix the asymmetric -ffast-math flags, remove hand-written C impostors, ensure same algorithm/workload/flags across all comparisons, report variance, and validate checksums.

2 MISSION

After execution: **(a)** the End driver (endc/driver/build.rs or equivalent) no longer adds -ffast-math to End while withholding it from C; both sides use identical optimization flags; **(b)** the benchmark suite's bench_*.c files that include windows.h, immintrin.h, or other headers endc cannot itself produce are removed or rewritten as actual End output; **(c)** every benchmark reports: algorithm name, workload size, warm-up count, repetition count, p50/p90/p99 latency, throughput, variance, output checksum; **(d)** the suite fails loudly if any of these fields is missing or zero; **(e)** an adversarial test: a benchmark with deliberately-wrong output checksum is detected and rejected.

3 CURRENT STATE

The End build driver adds -ffast-math to End-compiled binaries. The C side does not receive this flag. Several "End" benchmark sources are actually hand-written C with SIMD intrinsics that endc cannot itself emit — these are imposter benchmarks that misrepresent End's actual codegen. The benchmark suite reports timing without variance, repetition, or checksum validation.

4 VERIFIED PROBLEMS

- **P1**: Asymmetric -ffast-math — verified by reading the driver and the Python benchmark runner.
- **P2**: Hand-written C impostors in bench_*.c with <immintrin.h>.
- **P3**: No variance reporting.
- **P4**: No checksum validation — a benchmark that produces wrong output can still report a "fast" time.

5 NON-GOALS

- Designing new benchmarks — only fix the methodology of existing ones.
- Micro-optimizing End — the goal is honest measurement, not faster code.

6 PRESERVATION REQUIREMENTS

- The benchmark suite's directory structure is preserved.
- Existing benchmark programs that are honest End source continue to be used.

7 ARCHITECTURAL REQUIREMENTS

Define a benchmark manifest format TOML: `name, algorithm, workload_size, warmup_reps, measurement_reps, end_flags, c_flags, expected_output_sha256`. The runner reads the manifest, compiles both End and C versions with identical flags (modulo the language difference), runs warmup, runs N reps, computes p50/p90/p99, validates the output checksum, and exits non-zero if any field is missing or if the checksum mismatches. Imposter benchmarks (where the "End" source is actually hand-written C) are detected by requiring the End source to be compiled through endc and the resulting C to be diffed against a recorded baseline; if the diff is non-trivial (the C was hand-edited), the benchmark is rejected.

8 IMPLEMENTATION PLAN

1. **Audit** the driver: find every site where flags are added to End but not C, or vice versa.
2. **Unify** the flags: both sides get -O3 or both get -O2; -ffast-math either both or neither.
3. **Inventory** bench_*.c files: find every one that includes <immintrin.h>, <windows.h>, or other headers endc cannot produce.
4. **Rewrite** imposter benchmarks as End source that endc compiles — or remove them.
5. **Implement** the benchmark manifest format and runner.
6. **Add** checksum validation to every benchmark.
7. **Add** variance reporting.
8. **Write** adversarial test: a benchmark with wrong checksum is rejected.

9 FILE / MODULE INVESTIGATION

Inspect: endc/driver/build.rs (or wherever the GCC invocation is constructed), the Python runner run_suite12.py (or whatever it's named), benchmarks/ directory, every bench_*.c file. The agent must produce an inventory of imposter benchmarks.

10 DEPENDENCY REQUIREMENTS

No new crates.

11 DATA / API / PROTOCOL CONTRACTS

Benchmark result JSON: `{"name":"fib","algorithm":"recursive","workload_size":40,"warmup_reps":3,"measurement_reps":10,"end_flags":["-O3"],"c_flags":["-O3"],"latency_ms":{"p50":1234,"p90":1300,"p99":1450},"variance":0.04,"output_sha256":"abc...","expected_sha256":"abc...","checksum_match":true}`.

12 TEST PLAN

- **Flag symmetry**: grep the driver for -ffast-math — either zero matches or matches for both End and C paths.
- **Imposter detection**: every bench_*.c is regenerated from endc; the regenerated C is diffed against the committed C; non-empty diffs are flagged.
- **Checksum validation**: a benchmark with wrong expected checksum fails.
- **Variance reporting**: variance field is present and non-zero in every result.
- **Reproducibility**: same benchmark run twice, p50 within 10%.

13 EXECUTION TESTS

- grep -rn '\-ffast\-math' endc/driver/ benchmarks/ — either zero matches or symmetric.
- grep -rn 'immintrin\|windows\.h' benchmarks/ — zero matches.
- cargo run --bin bench_runner --bench fib — produces a complete JSON report with all fields.
- Adversarial: corrupt the expected checksum; rerun — fails.

14 FAILURE LOOP

If a benchmark's output checksum mismatches: STOP → either the implementation is wrong or the expected checksum is wrong → determine which by independent computation → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Adding -ffast-math to one side only.
- **FORBIDDEN**: Hand-writing C and presenting it as End output.
- **FORBIDDEN**: Disabling checksum validation.
- **FORBIDDEN**: Marking a benchmark as "approximate" to skip the checksum.

16 QUALITY GATES

- **Gate 1**: Flag symmetry — no asymmetric -ffast-math.
- **Gate 2**: Zero imposter benchmarks.
- **Gate 3**: Every benchmark produces a complete JSON report.
- **Gate 4**: Adversarial checksum test passes.

17 DEFINITION OF DONE

- Flag symmetry enforced.
- Imposter benchmarks removed.
- Manifest format + runner implemented.
- Checksum validation works.
- Variance reporting works.

18 FINAL AUDIT

- **Implementation audit**: no asymmetric flags, no impostors.
- **Runtime audit**: benchmarks produce real measurements.
- **Evidence audit**: Final Evidence contains a complete benchmark JSON report.

19 FINAL EVIDENCE

- Paste grep -rn '\-ffast-math' endc/driver/ benchmarks/ output (symmetric or zero).
- Paste grep -rn 'immintrin\|windows\.h' benchmarks/ output (zero matches).
- Paste a complete benchmark JSON report.
- Paste adversarial checksum-test failure output.

PROMPT 28

Documentation / CLI / VS Code Extension Truthfulness

Phase: 9 | **Priority: P1** | **Effort: 5 days** | **Depends: 01, 26**

Features: F-36, F-37, F-39

1 TITLE

Documentation / CLI / VS Code Extension Truthfulness — make every README claim match the implementation; fix the broken app_template.html; ship a basic LSP for the VS Code extension.

2 MISSION

After execution: **(a)** every claim in README.md, INSTALL.md, PRODUCTION_READINESS.md, RELEASE_NOTES_*.md, AGENTS.md is either backed by an executed example in the test suite or softened to "planned" / "experimental"; **(b)** every code example in the documentation is extracted into a test that runs it; **(c)** the broken app_template.html is either committed or the include_str! is removed (Prompt 01 should have fixed this — verify and document here); **(d)** the VS Code extension gains a basic LSP (using tower-lsp or lsp-types) supporting: go-to-definition, hover, diagnostics; **(e)** the README's "production-ready" claim is replaced by a per-subsystem status table reflecting the Capability Matrix (Part B of this document).

3 CURRENT STATE

After Prompt 01, the silent-zero fallback is removed and the build works. README still claims many features the implementation does not have. PRODUCTION_READINESS.md exists but overclaims. VS Code extension only does syntax highlighting.

4 VERIFIED PROBLEMS

- **P1**: README overclaims.
- **P2**: Documentation examples are not tested.
- **P3**: No LSP.

5 NON-GOALS

- Full IDE features (debugger integration, refactoring, etc.) — only basic LSP.
- Marketing the project — the goal is honesty, not promotion.

6 PRESERVATION REQUIREMENTS

- Existing documentation structure preserved; only the claims are corrected.

7 ARCHITECTURAL REQUIREMENTS

Documentation tests: a new endc/tests/docs_examples.rs extracts every fenced code block from README.md and runs it through the compiler; any example that fails to compile or produces wrong output fails the test. Add a per-subsystem status table to README: `| Subsystem | Status | Integration Test |`. VS Code LSP: add tower-lsp = "0.x", implement textDocument/definition, textDocument/hover, textDocument/publishDiagnostics.

8 IMPLEMENTATION PLAN

1. **Inventory** every claim in the docs.
2. **For each claim**, either back it with a test or soften it to "planned".
3. **Extract** every code example into a test.
4. **Add** the per-subsystem status table to README.
5. **Implement** basic LSP in the VS Code extension.
6. **Wire** the LSP to the compiler's diagnostic accumulator from Prompt 01.

9 FILE / MODULE INVESTIGATION

Inspect: every .md file in the repository, the VS Code extension's extension.ts (or equivalent).

10 DEPENDENCY REQUIREMENTS

Add: tower-lsp = "0.x", lsp-types = "0.x".

11 DATA / API / PROTOCOL CONTRACTS

Per-subsystem status table format in README: `| TLS | REAL | tests/stdlib/tls_integration.end | rustls |`.

12 TEST PLAN

- Documentation examples test: every fenced code block in README compiles and produces the documented output.
- Per-subsystem status table is consistent with the Capability Matrix in Part B.
- LSP: go-to-definition works for a simple example; hover shows type info; diagnostics appear in the editor.

13 EXECUTION TESTS

- cargo test --test docs_examples — all README examples pass.
- Open VS Code with the extension; open a .end file; hover over a variable — type info appears; introduce a syntax error — red squiggly appears.

14 FAILURE LOOP

If a README example fails: STOP → either the example is wrong or the implementation is wrong → fix at the right layer → re-run. If the LSP crashes: STOP → reproduce → fix → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Softening a claim to "planned" when the implementation is real (just to avoid writing a test).
- **FORBIDDEN**: Skipping the docs-examples test.
- **FORBIDDEN**: Leaving the per-subsystem table incomplete.

16 QUALITY GATES

- **Gate 1**: Docs examples test passes.
- **Gate 2**: Per-subsystem status table is complete and consistent with Part B.
- **Gate 3**: LSP provides the three basic operations.

17 DEFINITION OF DONE

- All README claims are backed by tests or marked "planned".
- Per-subsystem table in README.
- Docs examples test passes.
- VS Code LSP works.

18 FINAL AUDIT

- **Documentation audit**: every claim has evidence or is marked planned.
- **Test audit**: docs examples test passes.
- **Tooling audit**: LSP works.

19 FINAL EVIDENCE

- Paste cargo test --test docs_examples output.
- Paste the per-subsystem status table from README.
- Paste a screenshot or text description of LSP hover + diagnostics in VS Code.

PROMPT 29

Production Readiness & CI Gates

Phase: 10 | **Priority: P0** | **Effort: 7 days** | **Depends: 01-28** | **Features: F-38**

1 TITLE

Production Readiness & CI Gates — wire every quality gate from every previous prompt into a CI pipeline that refuses to merge a PR that fails any gate; publish a "production readiness" status report automatically on every release.

2 MISSION

After execution: **(a)** a CI pipeline (GitHub Actions) runs every quality gate from every prompt on every PR; **(b)** the pipeline is the only path to merge — no manual overrides; **(c)** the pipeline publishes a "Production Readiness Report" as a release artifact, listing per-subsystem status, per-gate status, and the count of passing/failing/ignored tests; **(d)** a "release" requires the pipeline to be green; **(e)** the pipeline includes: cargo test, golden_runner, stdlib_matrix_runner, bench_runner (with checksum validation), differential tests (interpreter vs. native, C backend vs. LLVM vs. WASM where applicable), adversarial tests, fuzz tests (where available), and a final audit script that reads every prompt's Final Evidence section and verifies the listed commands were actually executed.

3 CURRENT STATE

After all previous prompts, the gates exist but are not wired into CI. There is no enforced merge gate. Releases are manual.

4 VERIFIED PROBLEMS

- **P1**: No CI enforcement — a fake feature can be re-introduced.
- **P2**: No release-time readiness report.

5 NON-GOALS

- Distributed CI — single GitHub Actions runner is sufficient.
- Custom CI infrastructure — use GitHub Actions.

6 PRESERVATION REQUIREMENTS

- All gates from previous prompts continue to exist and run.

7 ARCHITECTURAL REQUIREMENTS

A new .github/workflows/ci.yml defines jobs: cargo_test, golden_runner, stdlib_matrix, bench_integrity, differential_tests, adversarial_tests, docs_examples, final_evidence_audit. The final_evidence_audit job runs a script that parses every prompt's Final Evidence section (in this audit document), executes the listed commands, and verifies the listed outputs match. A new .github/workflows/release.yml triggers on tag push, runs the full pipeline, and publishes a Release Report artifact. Branch protection rules require the ci.yml jobs to pass before merge.

8 IMPLEMENTATION PLAN

1. **Write** .github/workflows/ci.yml with all jobs.
2. **Write** the final_evidence_audit script (in Python or Rust).
3. **Write** .github/workflows/release.yml.
4. **Configure** branch protection rules on GitHub (manual step, document in INSTALL.md).
5. **Run** the full pipeline on the main branch; fix any failures.
6. **Publish** the first Production Readiness Report.

9 FILE / MODULE INVESTIGATION

Inspect: existing .github/workflows/ directory.

10 DEPENDENCY REQUIREMENTS

System: GitHub Actions runner. Optional: Docker for tests that need real PostgreSQL/Redis (Prompts 18, 19).

11 DATA / API / PROTOCOL CONTRACTS

Production Readiness Report JSON: `{"release":"v3.0.0","pipeline":"green","subsystems":[{"name":"TLS","status":"REAL","gate_pass":true},...],"tests":{"total":234,"passing":234,"failing":0,"ignored":0},"adversarial_tests_passing":12,"differential_tests_passing":200,"evidence_audit_passing":true}` .

12 TEST PLAN

- The CI pipeline runs on a PR that introduces a fake feature — the pipeline must fail.
- The CI pipeline runs on a PR that fixes a bug — the pipeline must pass.
- The release report is generated and attached to the GitHub release.

13 EXECUTION TESTS

- Open a PR that re-introduces the silent-zero fallback (e.g., `_ => "0".to_string())`; CI must fail.
- Open a PR that fixes a real bug; CI must pass.
- Push a tag; the release workflow runs and publishes the report.

14 FAILURE LOOP

If CI passes despite a fake feature: STOP → the gate is missing → add a test that would have caught the fake → re-run.

15 ANTI-CHEATING RULES

- **FORBIDDEN**: Marking a job as continue-on-error: true.
- **FORBIDDEN**: Disabling branch protection to merge a failing PR.
- **FORBIDDEN**: Skipping the final evidence audit.

16 QUALITY GATES

- **Gate 1**: CI pipeline runs end-to-end on every PR.
- **Gate 2**: Fake-feature PR fails.
- **Gate 3**: Real-bug-fix PR passes.
- **Gate 4**: Release report is generated and attached.
- **Gate 5**: Branch protection rules enforced (documented if manual).

17 DEFINITION OF DONE

- CI pipeline runs every gate.
- Fake-feature PR fails.
- Release report published.
- Branch protection enforced.

18 FINAL AUDIT

- **Requirement audit**: every gate is wired.
- **Implementation audit**: no continue-on-error.
- **Evidence audit**: Final Evidence contains the URL of a passing CI run and the URL of a failing fake-feature PR.

19 FINAL EVIDENCE

- Paste the URL of a green CI run.
- Paste the URL of a failing fake-feature PR.
- Paste the Production Readiness Report JSON for a release.

PART D

D. Master Execution Order

This part sequences the 29 prompts into 11 phases (Phase 0 through Phase 10), each with explicit **Prerequisites**, **Tasks** (referencing Prompt IDs from Part C), **Validation**, **Exit Gate**, and **Artifacts**. The phases are strictly ordered: a phase may not begin until the previous phase's Exit Gate is green. The only allowed parallelism is within a phase (multiple independent prompts may execute concurrently). The total estimated solo-developer time for Phases 0-4 is approximately 4-6 months; Phases 5-7 add another 4-6 months; Phases 8-10 add a final 2-3 months. The realistic timeline from start to "production-ready v3.0" is 12-15 months of focused solo work, longer if the developer is part-time.

Phase 0 — Baseline / Truth

P0

PREREQUISITES	Fresh clone of the repository at the audit commit.
TASKS	**Prompt 01** (Compiler Baseline & Truthfulness Gate), **Prompt 02** (End-to-C Golden Integration Verification System). These two prompts run in sequence — 01 first because 02 needs the diagnostics framework.
VALIDATION	Fresh-clone build succeeds. Catch-all silent zero is gone. `Diagnostic` struct exists. Golden runner exists with 50+ tests. The Range bug test fails (correctly) — this is the expected state at end of Phase 0.
EXIT GATE	All 5 quality gates from Prompt 01 pass. All 5 quality gates from Prompt 02 pass (including the deliberate Range-bug failure). **Block progression to Phase 1 until all gates are green.**
ARTIFACTS	`endc/src/diagnostics/mod.rs`, `endc/tests/golden/` with 50+ tests, `endc/tests/golden_runner.rs`.

Phase 1 — Compiler Correctness

P1

PREREQUISITES	Phase 0 Exit Gate green.
TASKS	**Prompt 03** (Semantic Type System & Error Handling) and **Prompt 04** (Parser Silent-Discard Elimination) may run in parallel. **Prompt 05** (C Backend Correctness) depends on both 03 and 04. **Prompt 06** (Compiler Regression Matrix) depends on 02, 03, 04, 05.
VALIDATION	All 200+ golden tests pass. The Range bug is fixed (the previously-failing test now passes). The enum codegen bug is fixed. All generated C is GCC-clean at `-Wall -Werror`. Differential testing passes.
EXIT GATE	`cargo run --bin golden_runner` exits zero with 200+ passing tests. **Block progression to Phase 2 until this is true.**
ARTIFACTS	`endc/src/semantic/types.rs` (with Type enum), `endc/src/semantic/borrow.rs`, refactored parser with `Result<AST, Diagnostic>`, fixed `stmt_gen.rs` and `expr_gen.rs`, `endc/tests/golden/_matrix.yaml`.

Phase 2 — Agentic Verification (Strategic Moat)

P2

PREREQUISITES	Phase 1 Exit Gate green.
TASKS	**Prompt 07** (Agent Contract System Foundation), **Prompt 08** (Agent Evidence & Verification Loop). 08 depends on 07.
VALIDATION	Contracts are real (TOML schema, lifecycle, verifier). Evidence is tamper-evident (HMAC-signed). STALE detection works. Security boundaries enforced. Repair loop produces structured feedback. 25+ end-to-end contract tests pass.
EXIT GATE	All 5 gates from Prompt 07 and all 6 gates from Prompt 08 pass. **This phase delivers the project's strategic moat — do not progress until it is solid.**
ARTIFACTS	`endc/src/agent/{contract,lifecycle,verifier,evidence,provenance,stale,signing}.rs`, `.agents/contract.toml` schema, `.agents/evidence/<contract_id>.json` bundles.

Phase 3 — Formal Verification

P3

PREREQUISITES	Phase 2 Exit Gate green.
TASKS	**Prompt 09** (SMT/Formal Verification with real Z3).
VALIDATION	Real Z3 integration. `proven += 1` removed. Adversarial tests: false properties → counterexample; true properties → verified; unsupported → UNKNOWN. TIMEOUT and SOLVER_ERROR distinguished from VERIFIED.
EXIT GATE	All 4 gates from Prompt 09 pass. The contract system can now include formal obligations as `required_tests`.
ARTIFACTS	`endc/src/semantic/smt_encode.rs`, Z3 in `Cargo.toml`, 20+ SMT tests.

Phase 4 — Real Backends

P4

PREREQUISITES	Phase 1 Exit Gate green (Phase 2 and 3 may proceed in parallel).
TASKS	**Prompt 10** (Cranelift/JIT), **Prompt 11** (LLVM), **Prompt 12** (WASM). All three depend only on 02 and 05 and may run in parallel.
VALIDATION	Cranelift: real executable memory, real function invocation, fib(20)=6765. LLVM: real `inkwell` IR, `opt` / `llc` pipeline, differential test against C. WASM: real `wabt` + `wasmtime`, differential test against C.
EXIT GATE	All gates from Prompts 10, 11, 12 pass. Differential tests across all four backends (C, Cranelift, LLVM, WASM) agree on all deterministic programs.
ARTIFACTS	Real `cranelift_backend.rs`, real `llvm_*.rs` using `inkwell`, real `wasm_backend.rs` using `wabt` + `wasmtime`.

Phase 5 — Security / Protocols

P5

PREREQUISITES	Phase 1 Exit Gate green.
TASKS	**Prompt 13** (TLS), **Prompt 14** (Cryptography/Argon2/HMAC), **Prompt 17** (SQLite), **Prompt 18** (PostgreSQL), **Prompt 19** (Redis), **Prompt 20** (HTTP/2 + HPACK), **Prompt 21** (Atomics/Mutex). All may run in parallel after Phase 1; they have no inter-prompt dependencies.
VALIDATION	All `is_connected = true`, `bytes_encrypted++`, `proven += 1`-style fake patterns are gone. Real libraries are in `Cargo.toml`. Interop tests with external tools (`curl`, `openssl`, `sqlite3`, `psql`, `redis-cli`) pass. Adversarial security tests pass (MITM, expired cert, tamper).
EXIT GATE	All gates from Prompts 13, 14, 17, 18, 19, 20, 21 pass. No fake patterns remain in `endc/src/`.
ARTIFACTS	Real `tls_tensor_canvas.rs` using `rustls`; real `concurrency_crypto.rs` using `argon2` + `hmac`; real SQLite/Postgres/Redis modules; real HTTP/2; TSan integration.

Phase 6 — AI / GPU / Distributed

P6

PREREQUISITES	Phase 5 Exit Gate green (for Raft which needs SQLite); Phase 1 for the others.
TASKS	**Prompt 15** (GGUF/AI Runtime), **Prompt 16** (GPU), **Prompt 22** (Raft), **Prompt 23** (Profiler), **Prompt 24** (Simulate/Stress), **Prompt 25** (Attestation).
VALIDATION	Real inference (candle) verified against `llama.cpp`. Real GPU execution (wgpu) verified against CPU reference. Real Raft cluster (3 nodes, leader election, replication, partition). Real profiler (differential test). Real stress (differential test). Real attestation (tamper test).
EXIT GATE	All gates from Prompts 15, 16, 22, 23, 24, 25 pass. No fake patterns remain anywhere.
ARTIFACTS	Real candle/wgpu/openraft/pprof/reqwest/tpm2 integrations.

Phase 7 — Standard Library Recovery

P7

PREREQUISITES	Phase 5 Exit Gate green (and Phase 6 for any stdlib module that depends on Raft/GPU/AI).
TASKS	**Prompt 26** (Standard Library Recovery).
VALIDATION	Every `std/*.end` file is classified. Every REAL module has an integration test. HTTP/1.x is real with conformance tests. EXPERIMENTAL modules are marked honestly.
EXIT GATE	All 3 gates from Prompt 26 pass. `stdlib_matrix.yaml` is complete. No facade claims REAL status without a test.
ARTIFACTS	`endc/tests/stdlib_matrix.yaml`, `stdlib_matrix_runner.rs`, HTTP conformance suite.

Phase 8 — Benchmark Integrity

P8

PREREQUISITES	Phase 1 Exit Gate green.
TASKS	**Prompt 27** (Benchmark Methodology & Integrity).
VALIDATION	No asymmetric `-ffast-math`. No imposter benchmarks. Every benchmark produces a complete JSON report with checksum validation. Adversarial checksum test passes.
EXIT GATE	All 4 gates from Prompt 27 pass. The README may now honestly claim "C performance with better DX" backed by reproducible numbers.
ARTIFACTS	Benchmark manifest format, fixed driver, fixed runner, checksum validation.

Phase 9 — Documentation & Tooling

P9

PREREQUISITES	Phases 0-8 largely complete (some claims cannot be backed by tests until the relevant subsystems are real).
TASKS	**Prompt 28** (Documentation / CLI / VS Code Extension Truthfulness).
VALIDATION	Every README claim is backed by a test or marked "planned". Per-subsystem status table is consistent with the Capability Matrix. Docs examples test passes. Basic LSP works.
EXIT GATE	All 3 gates from Prompt 28 pass.
ARTIFACTS	Honest README, per-subsystem status table, docs examples test, VS Code LSP.

Phase 10 — Production Readiness Gate

P10

PREREQUISITES	All previous phases green.
TASKS	**Prompt 29** (Production Readiness & CI Gates).
VALIDATION	CI pipeline runs every gate on every PR. Fake-feature PR fails. Real-bug-fix PR passes. Release report is published as an artifact. Branch protection is enforced.
EXIT GATE	All 5 gates from Prompt 29 pass. **End is now production-ready for the subsystems marked REAL in Part B.**
ARTIFACTS	`.github/workflows/ci.yml`, `.github/workflows/release.yml`, `final_evidence_audit` script, Production Readiness Report.

Ultimate Success Criterion (per directive)

When all phases pass, an external expert must be able to: clone the repo, run cargo run --bin golden_runner, run cargo run --bin stdlib_matrix_runner, run endc contract verify on a real contract, execute the binaries, inspect the generated C, challenge the claimed features, intentionally trigger failures, and independently conclude: **"This implementation is real, the claimed behavior is reproducible, and the system has mechanisms that make false success difficult."** That is the standard. Nothing less.

END OF MASTER AUDIT

Make it undeniable.

This document is a recovery program, not a bug-fix list. Every prompt is a contract with a weaker coding agent that demands real execution, real evidence, and real failure handling. The end state is a system where false success is mechanically difficult to introduce and even harder to maintain.

> **"This implementation is real, the claimed behavior is reproducible, and the system has mechanisms that make false success difficult."**

Begin with Phase 0. Do not skip quality gates. Do not compress prompts to save tokens. Do not declare a feature complete without Final Evidence. The standard is the standard.